

MULTI-MODEL NEURAL CDE INTERPRETABILITY

+ PHYSICS COMPARISON AUDIT

MATHEMATICS + PHYSICS SPECIFICATION (PRECISION REFERENCE)

Technical Abstract / Executive Summary

This report presents a **multi-model interpretability and stability audit** for Neural Controlled Differential Equation (Neural CDE) checkpoints used within the CondorBrain trading-signal architecture. The audit compares models along four orthogonal axes: (1) **feature attribution consistency**, (2) **output-space stability**, (3) **gradient-flow and saliency structure**, and (4) **loss-surface geometry** captured through Fisher Information and Hessian spectral analysis. Together, these dimensions provide a mathematically grounded characterization of whether two CDE checkpoints encode *the same functional mapping* or whether **logic drift** has occurred during training.

Permutation importance, SHAP approximations, gradient saliency, and surrogate regression trees are used to estimate and cross-validate the learned vector field

$$f(h_t, X_t)$$

that propagates the latent state through the CDE dynamics. Divergence between models' attribution distributions is quantified using cosine similarity, Jensen–Shannon divergence, Wasserstein distance, and rank-based correlations. Output stability is evaluated head-wise using correlation, mean squared deviation, variance ratios, Kolmogorov–Smirnov statistics, Wasserstein distance, and energy-based metrics.

To characterize the underlying optimization geometry, the audit computes **Fisher Information diagonals**, **Hessian eigenspectra**, and alignment scores between gradient saliency patterns. These metrics collectively reveal whether the models occupy **similar basins**, **adjacent basins**, or **distinct, sharp regions** of the loss landscape. Pairwise stability interpretations are expressed through physics analogies—coherence, energy gaps, amplitude stability, and distributional shift—providing intuitive diagnostics for structural model deviations.

Finally, a deterministic rubric transforms quantitative indicators into **pros, cons, and warnings** for each model, producing a ranked recommendation. This ensures that model selection is driven by mathematically interpretable properties rather than aggregate performance alone, enabling transparent, regulator-ready justification of model evolution and checkpoint selection.

0) PURPOSE

No equations in this section, but rewritten concisely:

The audit compares two or more Neural CDE checkpoints under a strict mathematical I/O contract. It evaluates:

- Feature attribution per model
- Extracted surrogate rules
- Divergences in outputs and attributions
- Dynamical stability vs. collapse/explosion
- Objective ranking and model selection

All metrics, preprocessing, and outputs must be deterministic and reproducible.

1) DEFINITIONS AND CORE OBJECTS

1.1 Dataset and Sequence Construction

Raw Time-Series Representation

$$X_{\text{raw}}[t] \in \mathbb{R}^D, t = 1, \dots, N$$

Symbol Definitions

- $X_{\text{raw}}[t]$: Feature vector at time index t .
 - D : Number of canonical input features.
 - N : Total number of time steps in the dataset.
-

Sequence Window Extraction

$$X_{\text{seq}}(i) = (X_{\text{raw}}[i], X_{\text{raw}}[i + 1], \dots, X_{\text{raw}}[i + T - 1]) \in \mathbb{R}^{T \times D}$$

Symbol Definitions

- $X_{\text{seq}}(i)$: Sequence window starting at index i .
 - T : Model-defined sequence length.
 - D : Dimensionality of each time step.
-

Common Window Length Across Models

$$T_{\text{common}} = \min_k (T_k)$$

Symbol Definitions

- T_{common} : Enforced sequence length for consistent cross-model comparison.
 - T_k : Sequence length required by model k .
 - k : Model index.
 - $\min_k (\cdot)$: Minimum across all k .
-

1.2 Model Outputs and ROI Head

Model Forward Map

$$y_k(i) = M_k(X_{\text{seq}}(i)) \in \mathbb{R}^H$$

Symbol Definitions

- $y_k(i)$: Output vector of model k for sample i .
 - $M_k(\cdot)$: Neural CDE model k .
 - H : Number of output heads.
-

ROI Head Extraction

$$y_{k,\text{ROI}}(i) = y_k(i)[h_{\text{ROI}}]$$

Symbol Definitions

- $y_{k,\text{ROI}}(i)$: ROI scalar output for model k .
 - h_{ROI} : Index of the ROI head (default value 5).
-

1.3 Robust Canonical Scaling

Median Vector

$$\mu_j = \text{median}_t (X_{\text{raw}}[t, j])$$

Symbol Definitions

- μ_j : Median of feature j .
- $X_{\text{raw}}[t, j]$: Value of feature j at time t .

Median Absolute Deviation

$$\text{MAD}_j = \text{median}_t (|X_{\text{raw}}[t, j] - \mu_j|)$$

Symbol Definitions

- MAD_j : Median absolute deviation of feature j .
- μ_j : Median of feature j .

Robust Scale

$$s_j = 1.4826 \text{MAD}_j$$

Symbol Definitions

- s_j : Robust scale for feature j .
- MAD_j : Median absolute deviation.
- 1.4826: Gaussian consistency factor.

Scaling Transformation

$$z_j = \frac{x_j - \mu_j}{s_j + \varepsilon}$$

Symbol Definitions

- z_j : Scaled feature j .
 - x_j : Raw feature value.
 - μ_j : Median.
 - s_j : Robust scale.
 - ε : Numerical stability constant.
-

1.4 Leakage Masking

$$X_{\text{seq}}(i)[\tau, j] = 0, \forall j \in \mathcal{L}, \forall \tau = 1, \dots, T$$

Symbol Definitions

- $\mathcal{L}$: Set of leakage feature indices.
 - τ : Time index inside a sequence window.
 - T : Sequence length.
 - $X_{\text{seq}}(i)[\tau, j]$: Feature j at timestep τ .
-

1.5 Numerical Validity Mask

$$\text{isFinite}(a) = \begin{cases} 1, & a \notin \{\text{NaN}, +\infty, -\infty\} \\ 0, & \text{otherwise} \end{cases}$$

Symbol Definitions

- $\text{isFinite}(a)$: Indicator of numerical validity.
- a : Scalar or tensor.
- $\text{NaN}, \pm\infty$: Invalid numeric values.

2) PER-MODEL INTERPRETABILITY MODULES

(Single-Model Analytics)

For each model M_k , analyses are computed on a shared set of sampled indices $S \subseteq \{1, \dots, N - T\}$. Sampling is deterministic under a fixed seed.

Module A — Permutation Importance (Gradient-Free, Distribution-Robust)

A.1 Baseline Outputs

$$y_{\text{base}}(i) = M[\cdot](X_{\text{seq}}(i))$$

Symbol Definitions

- $y_{\text{base}}(i)$: Baseline model output for sample i .
- $M(\cdot)$: Neural CDE model under analysis.
- $X_{\text{seq}}(i)$: Input sequence for index i .

Validity Indicator

$$\text{valid}(i) = \text{isFinite}[\cdot](y_{\text{base}}(i))$$

Symbol Definitions

- $\text{valid}(i)$: Indicator that all heads of $y_{\text{base}}(i)$ are finite.
- $\text{isFinite}(\cdot)$: Numerical validity function from §1.5.

Baseline Non-Finite Rate

$$\text{NF}_{\text{base}} = 1 - \frac{\sum_{i \in S} \text{valid}(i)}{|S|}$$

Symbol Definitions

- NF_{base} : Fraction of samples producing non-finite baseline outputs.
 - S : Set of sampled sequence indices.
 - $|S|$: Cardinality of S .
-

A.2 Permutation Operator for Feature j

Given feature index j , form a permuted input by shuffling only feature j across samples:

$$X_{\text{perm}}(i)[\tau, j] = X_{\text{seq}}(\pi(i))[\tau, j], X_{\text{perm}}(i)[\tau, m] = X_{\text{seq}}(i)[\tau, m], \quad m \neq j$$

Symbol Definitions

- $X_{\text{perm}}(i)$: Sequence for sample i after permutation of feature j .
 - τ : Time coordinate within a sequence window.
 - m : Feature index.
 - $\pi(i)$: Random permutation over indices S .
-

A.3 Perturbed Outputs

$$y_{\text{perm}}(i, j) = M[\cdot](X_{\text{perm}}(i, j))$$

Symbol Definitions

- $y_{\text{perm}}(i, j)$: Output when feature j is permuted in sample i .
 - $X_{\text{perm}}(i, j)$: Permuted input sequence.
-

A.4 Impact Delta

Multi-Head Distance

$$d(y_1, y_2) = \frac{1}{H} \sum_{h=1}^H |y_1[h] - y_2[h]|$$

Symbol Definitions

- $d(y_1, y_2)$: Mean absolute deviation across heads.
 - H : Number of output heads.
 - $y_1[h], y_2[h]$: Head-specific outputs.
-

Per-Sample Impact

$$\Delta(i, j) = d(y_{\text{base}}(i), y_{\text{perm}}(i, j))$$

Symbol Definitions

- $\Delta(i, j)$: Output change for permuting feature j in sample i .
-

A.5 Raw Importance

$$I_{\text{raw}}(j) = \frac{\sum_{i \in S_{\text{valid}}(j)} \Delta(i, j)}{|S_{\text{valid}}(j)|}$$

Symbol Definitions

- $I_{\text{raw}}(j)$: Unnormalized importance score for feature j .
 - $S_{\text{valid}}(j)$: Samples where both baseline and permuted outputs are finite.
-

A.6 Normalized Importance

$$I_{\text{norm}}(j) = 100 \cdot \frac{I_{\text{raw}}(j)}{\sum_{m=1}^D I_{\text{raw}}(m)}$$

Symbol Definitions

- $I_{\text{norm}}(j)$: Normalized importance assigned to feature j .
 - D : Total number of features.
-

A.7 Repeated-Permutation Uncertainty

$$I_{\text{raw,mean}}(j) = \frac{1}{R} \sum_{r=1}^R I_{\text{raw},r}(j)$$
$$I_{\text{raw,std}}(j) = \sqrt{\frac{1}{R} \sum_{r=1}^R (I_{\text{raw},r}(j) - I_{\text{raw,mean}}(j))^2}$$

Symbol Definitions

- $I_{\text{raw},r}(j)$: Raw importance on repetition r .
- R : Number of permutation repetitions.
- $I_{\text{raw,mean}}(j)$: Mean importance across repeats.
- $I_{\text{raw,std}}(j)$: Standard deviation of importance.

Module B — Surrogate Decision Tree Rule Extraction

B.1 Last-Step Feature Vector

$$x_{\text{last}}(i) = X_{\text{seq}}(i)[T, :] \in \mathbb{R}^D$$

Symbol Definitions

- $x_{\text{last}}(i)$: Terminal time-step features for sample i .
- T : Window length.
- D : Number of features.

B.2 Tree Target

$$t(i) = y_{k,\text{ROI}}(i)$$

Symbol Definitions

- $t(i)$: ROI scalar target for surrogate training.

B.3 Surrogate Prediction

$$\hat{t}(i) = \text{Tree}(\cdot)(x_{\text{last}}(i))$$

Symbol Definitions

- $\hat{t}(i)$: Surrogate tree prediction for sample i .
 - $\text{Tree}(\cdot)$: Trained decision tree regressor.
-

B.4 Surrogate Fidelity (Coefficient of Determination)

$$\begin{aligned} SS_{\text{res}} &= \sum_i (t(i) - \hat{t}(i))^2 \\ SS_{\text{tot}} &= \sum_i (t(i) - \bar{t})^2 \\ R^2 &= 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} \end{aligned}$$

Symbol Definitions

- SS_{res} : Residual sum of squares.
 - SS_{tot} : Total variance.
 - $\bar{t}$: Mean of $t(i)$ over valid samples.
 - R^2 : Surrogate fidelity metric.
-

Module C — Gradient Saliency

C.1 Input Gradient Tensor

$$g(i) = \frac{\partial y_{k,\text{ROI}}(i)}{\partial X_{\text{seq}}(i)} \in \mathbb{R}^{T \times D}$$

Symbol Definitions

- $g(i)$: Gradient sensitivity tensor for sample i .
 - T : Window length.
 - D : Number of features.
-

C.2 Absolute Gradient

$$a(i)[\tau, j] = |g(i)[\tau, j]|$$

Symbol Definitions

- $a(i)[\tau, j]$: Absolute gradient magnitude at time τ and feature j .
-

C.3 Feature-Wise Saliency

$$S_{\text{feat}}(j) = \frac{1}{|S|} \sum_{i \in S} \left(\frac{1}{T} \sum_{\tau=1}^T a(i)[\tau, j] \right)$$

Symbol Definitions

- $S_{\text{feat}}(j)$: Average saliency for feature j .
-

C.4 Temporal Saliency

$$S_{\text{time}}(\tau) = \frac{1}{|S|} \sum_{i \in S} \left(\frac{1}{D} \sum_{j=1}^D a(i)[\tau, j] \right)$$

Symbol Definitions

- $S_{\text{time}}(\tau)$: Sensitivity at timestep τ .
-

C.5 Gradient Alignment Between Two Models

Flattened Gradients

$$\vec{g}_A(i), \vec{g}_B(i) \in \mathbb{R}^{TD}$$

Symbol Definitions

- $\vec{g}_A(i)$: Flattened gradient for model A .
 - $\vec{g}_B(i)$: Flattened gradient for model B .
-

Cosine Alignment

$$\text{Align}(i) = \frac{\vec{g}_A(i)^\top \vec{g}_B(i)}{\|\vec{g}_A(i)\| \|\vec{g}_B(i)\| + \varepsilon}$$

Symbol Definitions

- $\text{Align}(i)$: Per-sample gradient alignment measure.
 - $\|\cdot\|$: Euclidean norm.
 - ε : Stability constant.
-

Average Alignment

$$\text{Align}_{\text{mean}} = \frac{1}{|S|} \sum_{i \in S} \text{Align}(i)$$

Symbol Definitions

- $\text{Align}_{\text{mean}}$: Expected cosine similarity across samples.
-

Module D — SHAP-Style Attribution (Ablation)

D.1 Baseline Output

$$y_{\text{base}}(i) = y_{k,\text{ROI}}(X_{\text{seq}}(i))$$

Symbol Definitions

- $y_{\text{base}}(i)$: Baseline ROI output.
-

D.2 Ablated Input

$$X_{-j}(i)[\tau, j] = r_j, X_{-j}(i)[\tau, m] = X_{\text{seq}}(i)[\tau, m] \quad (m \neq j)$$

Symbol Definitions

- $X_{-j}(i)$: Input sequence with feature j ablated.
 - r_j : Reference value (median or zero).
 - m : Feature index.
-

D.3 Ablated Output

$$y_{-j}(i) = y_{k,\text{ROI}}(X_{-j}(i))$$

Symbol Definitions

- $y_{-j}(i)$: ROI output when feature j is ablated.
-

D.4 Feature Attribution

$$\phi(i, j) = y_{\text{base}}(i) - y_{-j}(i)$$

Symbol Definitions

- $\phi(i, j)$: Attribution of feature j for sample i .

D.5 Magnitude Aggregation

$$\Phi(j) = \frac{1}{|S|} \sum_{i \in S} |\phi(i, j)|$$

Symbol Definitions

- $\Phi(j)$: Global attribution magnitude of feature j .

D.6 Normalization

$$\Phi_{\text{norm}}(j) = 100 \cdot \frac{\Phi(j)}{\sum_{m=1}^D \Phi(m)}$$

Symbol Definitions

- $\Phi_{\text{norm}}(j)$: Normalized SHAP-style importance.

3) PAIRWISE AND MULTI-MODEL COMPARISON METRICS

(Core Delta Analytics)

For K trained Neural CDE models, comparisons are computed pairwise. For any model pair (a, b) , let:

- I_a, I_b : normalized feature-importance vectors
- $Y_a(i), Y_b(i)$: ROI outputs across shared samples

All metrics below operate on aligned sequences under the shared window length T_{common} .

3.1 Divergence Metrics on Feature Importance Distributions

Let:

$$P_j = \frac{I_a(j)}{\sum_{m=1}^D I_a(m)}, Q_j = \frac{I_b(j)}{\sum_{m=1}^D I_b(m)}$$

Symbol Definitions

- P_j : Normalized importance distribution for model a .
 - Q_j : Normalized importance distribution for model b .
 - $I_a(j), I_b(j)$: Unnormalized importance scores.
 - D : Number of features.
-

3.1.1 Cosine Similarity

$$\cos(a, b) = \frac{\sum_{j=1}^D P_j Q_j}{\sqrt{\sum_{j=1}^D P_j^2} \sqrt{\sum_{j=1}^D Q_j^2 + \varepsilon}}$$

Symbol Definitions

- $\cos(a, b)$: Cosine similarity between importance distributions.
 - ε : Stability constant preventing division by zero.
-

3.1.2 Jensen–Shannon Divergence

$$\begin{aligned} M_j &= \frac{1}{2}(P_j + Q_j) \\ \text{KL}(P \parallel M) &= \sum_{j=1}^D P_j \log \left[\frac{P_j + \varepsilon}{M_j + \varepsilon} \right] \\ \text{KL}(Q \parallel M) &= \sum_{j=1}^D Q_j \log \left[\frac{Q_j + \varepsilon}{M_j + \varepsilon} \right] \\ \text{JSD}(P, Q) &= \frac{1}{2} \text{KL}(P \parallel M) + \frac{1}{2} \text{KL}(Q \parallel M) \end{aligned}$$

Symbol Definitions

- M_j : Mixture distribution.
 - $KL(P \parallel M)$: Kullback–Leibler divergence from P to M .
 - $JSD(P, Q)$: Jensen–Shannon divergence.
 - $\log$: Natural logarithm.
-

3.1.3 Spearman Rank Correlation

Let $r_a(j)$ and $r_b(j)$ denote the ranks of feature j under models a and b . Then:

$$d_j = r_a(j) - r_b(j)$$
$$\rho_{\text{Spearman}} = 1 - \frac{6 \sum_{j=1}^D d_j^2}{D(D^2 - 1)}$$

Symbol Definitions

- d_j : Rank difference for feature j .
 - ρ_{Spearman} : Spearman rank correlation coefficient.
-

3.1.4 Kendall Tau

$$\tau = \frac{N_{\text{con}} - N_{\text{dis}}}{N_{\text{tot}}}$$

Symbol Definitions

- N_{con} : Number of concordant feature pairs.
 - N_{dis} : Number of discordant feature pairs.
 - N_{tot} : Total number of feature pairs $\binom{D}{2}$.
 - τ : Kendall rank correlation.
-

3.1.5 Top- K Overlap

$$\text{Overlap}_K(a, b) = \frac{|\text{TopK}(a) \cap \text{TopK}(b)|}{K}$$

Symbol Definitions

- $\text{TopK}(a)$: Indices of top- K features under model a .
 - $\text{Overlap}_K(a, b)$: Fraction of shared top- K features.
-

3.2 Output Distribution Divergences

Let outputs be collected across shared samples:

$$Y_a(i), Y_b(i)$$

Symbol Definitions

- $Y_a(i)$: ROI output of model a .
 - $Y_b(i)$: ROI output of model b .
-

3.2.1 Mean Squared Difference (Energy Gap)

$$\text{MSD}(a, b) = \frac{1}{|S_{\text{valid}}|} \sum_{i \in S_{\text{valid}}} (Y_a(i) - Y_b(i))^2$$

Symbol Definitions

- $\text{MSD}(a, b)$: Mean squared difference between output fields.
 - S_{valid} : Samples where both outputs are finite.
-

3.2.2 Pearson Correlation (Coherence)

$$\bar{Y}_a = \frac{1}{|S|} \sum_{i \in S} Y_a(i), \bar{Y}_b = \frac{1}{|S|} \sum_{i \in S} Y_b(i)$$

$$\text{corr}(a, b) = \frac{\sum_{i \in S} (Y_a(i) - \bar{Y}_a) (Y_b(i) - \bar{Y}_b)}{\sqrt{\sum_{i \in S} (Y_a(i) - \bar{Y}_a)^2} \sqrt{\sum_{i \in S} (Y_b(i) - \bar{Y}_b)^2 + \varepsilon}}$$

Symbol Definitions

- $\bar{Y}_a, \bar{Y}_b$: Sample means.
 - $\text{corr}(a, b)$: Pearson correlation coefficient.
-

3.2.3 Variance Ratio (Amplitude Stability)

$$\text{VR}(a, b) = \frac{\text{Var}(Y_a)}{\text{Var}(Y_b) + \varepsilon}$$

Symbol Definitions

- $\text{VR}(a, b)$: Ratio of output variances.
 - Var : Sample variance operator.
-

3.2.4 Kolmogorov–Smirnov Statistic

$$\text{KS}(a, b) = \sup_x |F_a(x) - F_b(x)|$$

Symbol Definitions

- $F_a(x)$: Empirical CDF of Y_a .
 - $F_b(x)$: Empirical CDF of Y_b .
 - $\text{KS}(a, b)$: Kolmogorov–Smirnov distance.
-

3.2.5 Wasserstein–1 (Earth Mover Distance)

$$W_1(a, b) = \int_0^1 |F_a^{-1}(u) - F_b^{-1}(u)| du$$

Symbol Definitions

- $W_1(a, b)$: Wasserstein-1 distance.
 - F^{-1} : Quantile function.
 - u : Uniform CDF parameter.
-

3.2.6 Energy Distance

$$\text{ED}(a, b) = 2 \mathbb{E}[|X - Y|] - \mathbb{E}[|X - X'|] - \mathbb{E}[|Y - Y'|]$$

Symbol Definitions

- X, X' : i.i.d. samples from Y_a .
 - Y, Y' : i.i.d. samples from Y_b .
 - $\text{ED}(a, b)$: Energy distance.
-

3.2.7 Maximum Mean Discrepancy

$$\text{MMD}^2(a, b) = \mathbb{E}[k(X, X')] + \mathbb{E}[k(Y, Y')] - 2 \mathbb{E}[k(X, Y)]$$

Symbol Definitions

- $k(\cdot, \cdot)$: Positive-definite kernel.
 - $\text{MMD}(a, b)$: Maximum mean discrepancy.
-

3.3 Decision-Rule Divergence

Split-Set Intersection

$$\text{Agree}_d(a, b) = \frac{|\text{SplitSet}_d(a) \cap \text{SplitSet}_d(b)|}{|\text{SplitSet}_d(a) \cup \text{SplitSet}_d(b)|}$$

Symbol Definitions

- $\text{SplitSet}_d(a)$: Features used at depth d of model a 's surrogate tree.
 - $\text{Agree}_d(a, b)$: Depth-wise rule agreement.
-

Surrogate Fidelity Delta

$$\Delta R^2(a, b) = R_a^2 - R_b^2$$

Symbol Definitions

- R_a^2, R_b^2 : Surrogate fidelity scores.
- $\Delta R^2(a, b)$: Difference in interpretability.

4) PHYSICS-INSPIRED STABILITY & DYNAMICAL INTERPRETATION

Neural CDEs define hidden-state trajectories driven by a learned vector field. Instability manifests as collapse (vanishing variance), explosion (non-finite outputs), or chaotic sensitivity to small perturbations.

4.1 Continuous-Time Neural CDE Dynamics

A Neural CDE evolves latent state $h(t)$ via:

$$\frac{dh(t)}{dt} = f(\cdot)(h(t), t; \theta) \frac{dX(t)}{dt}$$

Symbol Definitions

- $h(t)$: Latent state at time t .
 - $f(\cdot)$: Learned vector field.
 - $X(t)$: Control path derived from input features.
 - θ : Model parameters.
 - $\frac{dh(t)}{dt}$: Continuous-time latent evolution.
-

4.2 Collapse & Explosion Diagnostics

4.2.1 Output Variance

$$\text{Var}(Y) = \frac{1}{|S|} \sum_{i \in S} (Y(i) - \bar{Y})^2$$

Symbol Definitions

- $\text{Var}(Y)$: Output variance.
 - $Y(i)$: ROI output of model on sample i .
 - $\bar{Y}$: Sample mean.
 - S : Evaluation sample set.
-

4.2.2 Non-Finite Rate

$$\text{NF} = \frac{\#\{i: Y(i) \text{ is non-finite}\}}{|S|}$$

Symbol Definitions

- NF : Fraction of outputs that are NaN or $\pm\infty$.
-

4.2.3 Tail Ratio (Asymmetry & Instability Signal)

Let:

$$p_q = \text{quantile}(Y, q)$$

Then:

$$\text{TailRatio} = \frac{p_{0.99} - p_{0.50}}{p_{0.50} - p_{0.01} + \varepsilon}$$

Symbol Definitions

- p_q : q -quantile of outputs.
 - TailRatio: Asymmetry measure; large values indicate heavy positive tails.
-

4.3 Sensitivity to Small Perturbations (Lyapunov-Style Proxy)

Let perturbed input sequences be:

$$X_{\text{pert}}(i) = X_{\text{seq}}(i) + \delta(i)$$

Symbol Definitions

- $X_{\text{pert}}(i)$: Perturbed input.
 - $\delta(i)$: Small additive noise tensor.
-

4.3.1 Perturbed Output

$$y_{\text{pert}}(i) = y_{k,\text{ROI}}(X_{\text{pert}}(i))$$

Symbol Definitions

- $y_{\text{pert}}(i)$: ROI output from perturbed sequence.
-

4.3.2 Local Sensitivity

$$S_i = \frac{|y_{\text{pert}}(i) - y_{\text{base}}(i)|}{\|\delta(i)\|_F + \varepsilon}$$

Symbol Definitions

- S_i : Sensitivity of sample i .

- $y_{\text{base}}(i)$: Baseline ROI output.
 - $\|\cdot\|_F$: Frobenius norm.
 - ε : Stability constant.
-

4.3.3 Mean Sensitivity

$$S_{\text{mean}} = \frac{1}{|S|} \sum_{i \in S} S_i$$

Symbol Definitions

- S_{mean} : Average perturbation sensitivity across samples.
-

4.3.4 Log-Growth (Lyapunov-Like) Indicator

$$L_i = \log \left(|y_{\text{pert}}(i) - y_{\text{base}}(i)| + \varepsilon \right) - \log \left(\|\delta(i)\|_F + \varepsilon \right)$$
$$L_{\text{mean}} = \frac{1}{|S|} \sum_{i \in S} L_i$$

Symbol Definitions

- L_i : Log-ratio of output displacement to input perturbation.
 - L_{mean} : Mean growth factor; positive values imply expansion.
-

4.4 Coherence–Energy Stability Map

4.4.1 Output Coherence

$$\text{Coh}(a, b) = \text{corr}(a, b)$$

Symbol Definitions

- $\text{Coh}(a, b)$: Output correlation (Section 3.2.2).

4.4.2 Output Energy Gap

$$\text{Eng}(a, b) = \text{MSD}(a, b)$$

Symbol Definitions

- $\text{Eng}(a, b)$: Mean squared difference (Section 3.2.1).

4.4.3 Phase Diagram Coordinates

Each model pair (a, b) is mapped to:

$$(\text{Coh}(a, b), \text{Eng}(a, b))$$

Interpretation

- High coherence + low energy: same learned dynamics.
- High coherence + high energy: amplitude drift.
- Low coherence + low energy: mild behavioral drift.
- Low coherence + high energy: regime shift or structural divergence.

5) MULTI-MODEL SUMMARY VIEWS ($K \geq 3$)

These views aggregate all pairwise metrics into full $K \times K$ matrices and provide global summaries of divergence and stability across all models.

5.1 Importance-Based Divergence Matrices

For models $1, \dots, K$, assemble:

$$\begin{aligned} \mathbf{C}_{\cos}[a, b] &= \cos(a, b) \\ \mathbf{C}_{\text{jSD}}[a, b] &= \text{JSD}(P^{(a)}, P^{(b)}) \\ \mathbf{C}_{\text{spr}}[a, b] &= \rho_{\text{Spearman}}(a, b) \\ \mathbf{C}_{\text{topK}}^{(K)}[a, b] &= \text{Overlap}_K(a, b) \end{aligned}$$

Symbol Definitions

- $\mathbf{C}_{\text{cos}}$: Cosine-similarity matrix.
 - $\mathbf{C}_{\text{jsd}}$: Jensen–Shannon divergence matrix.
 - $\mathbf{C}_{\text{spr}}$: Spearman rank-correlation matrix.
 - $\mathbf{C}_{\text{topK}}^{(K)}$: Top- K feature-overlap matrix.
 - a, b : Model indices.
-

5.2 Output-Based Stability Matrices

$$\begin{aligned}\mathbf{S}_{\text{corr}}[a, b] &= \text{corr}(a, b) \\ \mathbf{S}_{\text{msd}}[a, b] &= \text{MSD}(a, b) \\ \mathbf{S}_{\text{ks}}[a, b] &= \text{KS}(a, b) \\ \mathbf{S}_{\text{w1}}[a, b] &= W_1(a, b)\end{aligned}$$

Symbol Definitions

- $\mathbf{S}_{\text{corr}}$: Correlation (coherence) matrix.
 - $\mathbf{S}_{\text{msd}}$: Mean-squared-difference matrix.
 - $\mathbf{S}_{\text{ks}}$: Kolmogorov–Smirnov distance matrix.
 - $\mathbf{S}_{\text{w1}}$: Wasserstein-1 distance matrix.
-

5.3 Cross-Domain Matrices

For the gradient, Fisher, and Hessian comparisons:

$$\begin{aligned}\mathbf{G}[a, b] &= \text{Align}_{\text{mean}}(a, b) \\ \mathbf{F}[a, b] &= \text{cosine}[\cdot, \cdot](F^{(a)}, F^{(b)}) \\ \mathbf{H}[a, b] &= \text{dist}_{\text{eig}}[\cdot, \cdot](\lambda^{(a)}, \lambda^{(b)})\end{aligned}$$

Symbol Definitions

- $\mathbf{G}$: Gradient-alignment matrix.
- $\mathbf{F}$: Fisher-information similarity matrix.
- $\mathbf{H}$: Hessian-eigenspectrum divergence matrix.
- $F^{(a)}$: Fisher diagonal vector for model a .
- $\lambda^{(a)}$: Leading eigenvalues of Hessian for model a .

- dist_{eig} : Spectral distance metric.
-

6) RANKING AND BEST-MODEL SELECTION

Each model receives a quantitative score composed of stability, interpretability, agreement, and optional novelty components.

6.1 Stability Score

$$\text{StabilityScore}(k) = 1 - w_{\text{nf}} \text{NF}_k - w_{\text{col}} \text{Collapse}_k - w_{\text{exp}} \text{Explosion}_k - w_{\text{tail}} \text{TailSignal}_k$$

Symbol Definitions

- NF_k : Non-finite output rate.
 - Collapse_k : Collapse indicator (e.g., near-zero variance).
 - Explosion_k : Explosion indicator (e.g., high NF, large variance).
 - TailSignal_k : Tail-ratio abnormality.
 - w : Deterministic weighting constants.
-

6.2 Interpretability Score

$$\text{InterpScore}(k) = \alpha R_k^2 + \beta \text{Sparsity}_k$$

Symbol Definitions

- R_k^2 : Surrogate-tree fidelity for model k .
 - Sparsity_k : Number of distinct split features (penalizes overly complex rules).
 - α, β : Deterministic coefficients.
-

6.3 Agreement Score

$$\text{AgreeScore}(k) = \frac{1}{K-1} \sum_{\substack{b=1 \\ b \neq k}}^K (\gamma_1 \cos(k, b) + \gamma_2 \text{corr}(k, b) + \gamma_3 \text{Overlap}_{10}(k, b))$$

Symbol Definitions

- $\text{AgreeScore}(k)$: Similarity of model k to all peers.
 - γ_i : Fixed weights.
-

6.4 Novelty Score (Optional)

$$\text{NoveltyScore}(k) = 1 - \text{AgreeScore}(k)$$

Symbol Definitions

- Rewards models whose behavior diverges from peers.
-

6.5 Total Score

$$\begin{aligned} \text{TotalScore}(k) &= w_1 \text{StabilityScore}(k) + w_2 \text{InterpScore}(k) + w_3 \text{AgreeScore}(k) \\ &\quad + w_4 \text{NoveltyScore}(k) \end{aligned}$$

Symbol Definitions

- $\text{TotalScore}(k)$: Final ranking metric.
 - w_i : Weight parameters logged to JSON.
-

7) PLOTS AND CHARTS SPECIFICATION

No equations required; this section is descriptive.
Plots include:

- Bar charts for top features

- Tree-rule diagrams
 - Output histograms + distribution plots
 - K×K heatmaps for all divergence metrics
 - Radar charts for model scores
 - Scatter plots for coherence vs. energy
-

8) REPORT OUTPUTS (FILE CONTRACT)

No equations.

Defines the required artifacts:

- Markdown report
 - JSON structured outputs (scaling, metrics, rules, plots)
 - PNG plots
-

9) ENGINEERING CONTRACT & FAILURE MODES

Includes deterministic behaviors for:

- Sequence-length mismatches
- Non-finite outputs
- Missing features
- Leakage-column enforcement
- Gradient instability fallbacks
- Sanity checks and reproducibility

No new equations in this section.

10) IMPLEMENTATION BLUEPRINT (FORMAL SPECIFICATION)

This section contains algorithmic steps. Where math is required, equations appear below.

10.1 Sample Index Selection

$$S_{\text{common}} \subseteq \{ 1, 2, \dots, N - T_{\text{common}} \}$$

Symbol Definitions

- S_{common} : Deterministically sampled indices used for evaluation.
 - N : Number of dataset rows.
 - T_{common} : Common sequence length.
-

10.2 Sequence Construction

$$X_{\text{seq}}(i) = (X_{\text{raw}}[i], \dots, X_{\text{raw}}[i + T_{\text{common}} - 1])$$

(Already defined previously; included here for completeness.)

10.3 Model Evaluation

$$y_k(i) = M_k(\cdot)(X_{\text{seq}}(i))$$

Symbol Definitions

- $M_k(\cdot)$: Model k .
 - $y_k(i)$: Vector of outputs for sample i .
-

10.4 Global MI Computation (If Enabled)

Pairwise MI between features:

$$\text{MI}(j, k) = \iint p(x_j, x_k) \log \left(\frac{p(x_j, x_k)}{p(x_j)p(x_k)} \right) dx_j dx_k$$

Symbol Definitions

- x_j, x_k : Random variables corresponding to features j, k .
- $p(\cdot)$: PDF estimates.
- $\text{MI}(j, k)$: Mutual information.

11) PROS / CONS AUTO-GENERATION (DETERMINISTIC RUBRIC)

This section defines deterministic rule triggers for model-specific pros/cons. Only a few rules contain mathematics; all are formalized below.

11.1 Collapse Trigger

A model is flagged for collapse if:

$$\text{Var}(Y_k) \leq \tau_{\text{collapse}}$$

Symbol Definitions

- $\text{Var}(Y_k)$: Variance of ROI outputs for model k .
 - τ_{collapse} : Collapse threshold (small positive constant).
-

11.2 Explosion Trigger

$$\text{NF}_k \geq \tau_{\text{NF}} \text{ or } \text{Var}(Y_k) \geq \tau_{\text{var}}$$

Symbol Definitions

- NF_k : Non-finite output rate.
 - τ_{NF} : Allowed NF threshold.
 - τ_{var} : Variance explosion threshold.
-

11.3 Logic Drift Trigger

$$\rho_{\text{Spearman}}(k, k_{\text{ref}}) \leq \tau_{\text{rank}}$$

Symbol Definitions

- k_{ref} : Reference model index.

- ρ_{Spearman} : Rank correlation from Section 3.1.3.
 - τ_{rank} : Minimum acceptable agreement threshold.
-

11.4 Fidelity Trigger for Surrogate Trees

$$R_k^2 < \tau_{R^2}$$

Symbol Definitions

- R_k^2 : Surrogate-tree R^2 score for model k .
 - τ_{R^2} : Minimum fidelity threshold.
-

Pros and cons are generated by evaluating these inequalities deterministically.

12) STRONG ADDITIONS (REQUIRED EXTENSIONS)

These modules add deeper reliability, spectral, and regime-aware insights.

12.1 Calibration Metrics (If ROI Approximates a Probability)

12.1.1 Brier Score

$$\text{Brier} = \frac{1}{|S|} \sum_{i \in S} (p(i) - y(i))^2$$

Symbol Definitions

- $p(i)$: Model probability prediction.
- $y(i)$: Binary or continuous target.
- Brier: Calibration error score.

12.1.2 Expected Calibration Error (ECE)

Partition predictions into bins B_m . Then:

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{|S|} |\text{acc}(B_m) - \text{conf}(B_m)|$$

Symbol Definitions

- B_m : Bin m of predictions.
- $\text{acc}(B_m)$: Empirical accuracy of bin.
- $\text{conf}(B_m)$: Mean prediction confidence in bin.
- M : Number of bins.
- ECE: Expected calibration error.

12.2 Latent Embedding Spectral Drift

Given latent final states $h_T^{(k)}(i)$:

12.2.1 Covariance Estimate

$$\Sigma_k = \frac{1}{|S|} \sum_{i \in S} \left(h_T^{(k)}(i) - \bar{h}_T^{(k)} \right) \left(h_T^{(k)}(i) - \bar{h}_T^{(k)} \right)^\top$$

Symbol Definitions

- Σ_k : Covariance matrix of latent embeddings for model k .
- $\bar{h}_T^{(k)}$: Mean latent vector for model k .

12.2.2 Spectral Comparison

Let eigenvalues be sorted:

$$\lambda_1^{(k)} \geq \lambda_2^{(k)} \geq \dots \geq \lambda_d^{(k)}$$

Spectral drift is measured by:

$$\text{SpecDist}(a, b) = \sum_{j=1}^d \left(\lambda_j^{(a)} - \lambda_j^{(b)} \right)^2$$

Symbol Definitions

- $\lambda_j^{(k)}$: j -th eigenvalue of covariance Σ_k .
 - d : Latent dimensionality.
 - $\text{SpecDist}(a, b)$: Spectral drift between models.
-

12.3 Regime-Conditioned Comparisons

For regime label r :

$$\text{MSD}_r(a, b) = \frac{1}{|S_r|} \sum_{i \in S_r} (Y_a(i) - Y_b(i))^2$$

Symbol Definitions

- S_r : Samples belonging to regime r .
 - $\text{MSD}_r(a, b)$: Energy-gap restricted to regime r .
-

12.4 Temporal-Sensitivity Signature

$$\text{Sig}_k(\tau) = S_{\text{time}}^{(k)}(\tau)$$

Symbol Definitions

- $\text{Sig}_k(\tau)$: Per-timestep saliency signature for model k , from Section 2C.

13) CROSS-CHECKS, VALIDATION, & AUDIT TRAIL

Most of this section is logic, not equations.
The only mathematical components formalize contradictions and reproducibility.

13.1 Reproducibility Hash

Given feature vector list F :

$$\text{Hash}(F) = \text{SHA256}(\text{serialize}(F))$$

Symbol Definitions

- $\text{Hash}(F)$: Deterministic hash of features.
 - SHA256 : Cryptographic hash function.
 - $\text{serialize}(F)$: Ordered string encoding of the feature list.
-

13.2 Contradiction Indicator

A contradiction is flagged when:

$$\text{JSD}(P^{(a)}, P^{(b)}) \gg 0 \text{ but } \text{corr}(a, b) \approx 1$$

Symbol Definitions

- “ $\gg 0$ ” means significantly above expected noise floor.
 - Indicates feature-redundancy or reparameterization effects.
-

13.3 Permutation Sanity Check

$$I_{\text{sum}} = \sum_{j=1}^D I_{\text{raw}}(j)$$

If:

$$I_{\text{sum}} = 0$$

→ Model interpreted as degenerate or collapsed.

Symbol Definitions

- I_{sum} : Total raw permutation importance.

14) PHYSICS INTERPRETATION REQUIREMENTS

The following metrics must be mapped to dynamical-systems terminology.

14.1 Energy Gap

$$E(a, b) = \text{MSD}(a, b)$$

Symbol Definitions

- $E(a, b)$: Energy separation between scalar fields.
-

14.2 Coherence

$$C(a, b) = \text{corr}(a, b)$$

Symbol Definitions

- $C(a, b)$: Coherence measure (synchronization level).
-

14.3 Amplitude Stability

$$A(a,b) = \text{VR}(a,b)$$

Symbol Definitions

- $A(a,b)$: Amplitude-scaling stability ratio.
-

14.4 Sensitivity (Lyapunov Proxy)

$$\Lambda_k = S_{\text{mean}}(k)$$

Symbol Definitions

- Λ_k : Lyapunov-like stability proxy.
-

14.5 Signal-Structure Complexity

$$\text{Entropy}(P) = - \sum_{j=1}^D P_j \log (P_j)$$

Symbol Definitions

- $\text{Entropy}(P)$: Attribute-distribution entropy.
 - P_j : Normalized importance (Section 3).
-

15) APPENDIX — CANONICAL EQUATION LIBRARY

This appendix consolidates all base formulas used throughout the audit.

15A. Robust Scaling

$$z_j = \frac{x_j - \mu_j}{s_j + \varepsilon}$$

Symbol Definitions

- z_j : Scaled feature.
 - x_j : Raw feature value.
 - μ_j, S_j : Median and robust-scale parameters.
-

15B. Median Absolute Deviation

$$\text{MAD}_j = \text{median}_t |X_{\text{raw}}[t, j] - \mu_j|$$

Symbol Definitions

- MAD_j : Median absolute deviation.
-

15C. Permutation Importance

$$I_{\text{raw}}(j) = \frac{1}{|S_{\text{valid}}|} \sum_{i \in S_{\text{valid}}} \Delta(i, j)$$
$$I_{\text{norm}}(j) = 100 \cdot \frac{I_{\text{raw}}(j)}{\sum_{m=1}^D I_{\text{raw}}(m)}$$

Symbol Definitions

- $I_{\text{raw}}, I_{\text{norm}}$: Raw and normalized importance.
-

15D. Surrogate Tree Fidelity

$$R^2 = 1 - \frac{\sum_i (t(i) - \hat{t}(i))^2}{\sum_i (t(i) - \bar{t})^2}$$

Symbol Definitions

- $t(i), \hat{t}(i)$: True and surrogate-predicted ROI.

15E. Cosine Similarity

$$\cos(a, b) = \frac{\sum_j P_j Q_j}{\sqrt{\sum_j P_j^2} \sqrt{\sum_j Q_j^2 + \varepsilon}}$$

15F. Jensen–Shannon Divergence

$$\text{JSD}(P, Q) = \frac{1}{2} \text{KL}(P \parallel M) + \frac{1}{2} \text{KL}(Q \parallel M)$$

15G. Wasserstein Distance

$$W_1 = \int_0^1 |F_a^{-1}(u) - F_b^{-1}(u)| du$$

15H. Sensitivity Measure

$$S_i = \frac{|y_{\text{pert}}(i) - y_{\text{base}}(i)|}{\|\delta(i)\|_F + \varepsilon}$$

15I. Lyapunov Proxy

$$L_{\text{mean}} = \frac{1}{|S|} \sum_{i \in S} [\log(|y_{\text{pert}}(i) - y_{\text{base}}(i)| + \varepsilon) - \log(\|\delta(i)\|_F + \varepsilon)]$$

Appendix A: Python Script

```
1. """
2. Multi-Model CDE Interpretability Comparison Audit (Comprehensive Edition)
3.
4. Compares 2+ Neural CDE checkpoints with:
5. - Permutation importance rankings
6. - Surrogate decision tree analysis
7. - Mathematical divergence metrics (Cosine, JS, Spearman, Wasserstein)
8. - Physics-inspired stability analysis (Energy, Coherence, KS)
9. - Gradient alignment / saliency analysis
10. - Fisher Information estimation
11. - Hessian eigenspectrum (loss landscape curvature)
12. - SHAP-style attribution comparison
13. - Mutual Information analysis
14. - Comprehensive visualizations with interpretation
15.
16. Usage:
17.     python intelligence/audit_cde_comparison.py \
18.         --models models/epoch_1.pth/epoch_3.pth \
19.         --data data/processed/mamba_institutional_2024_1m_v22.csv \
20.         --samples 3000 \
21.         --output reports/model_comparison.md
22. """
23.
24. import torch
25. import torch.nn as nn
26. import torch.nn.functional as F
27. import numpy as np
28. import pandas as pd
29. import argparse
30. import sys
31. import os
32. import json
33. import warnings
34. from datetime import datetime
35. from collections import OrderedDict
36. from sklearn.tree import DecisionTreeRegressor, export_text, plot_tree
37. from sklearn.metrics import mean_squared_error, mutual_info_score
38. from sklearn.feature_selection import mutual_info_regression
39. from scipy import stats
40. from scipy.spatial.distance import cosine, jensenshannon
41. from scipy.stats import wasserstein_distance, entropy
42. from scipy.linalg import eigvalsh
43. from tqdm import tqdm
44.
45. import matplotlib
46. matplotlib.use('Agg')
47. import matplotlib.pyplot as plt
48. import matplotlib.gridspec as gridspec
49. from matplotlib.colors import LinearSegmentedColormap
50. import seaborn as sns
51.
52. warnings.filterwarnings('ignore')
53.
54. # Add project root to path
55. sys.path.insert(0, os.getcwd())
56.
57. from intelligence.condor_brain import CondorBrain
58. from intelligence.canonical_feature_registry import FEATURE_COLS_V22, INPUT_DIM_V22
59.
60. DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
61. EPS = 1e-6
62.
63. # Output head names for interpretation
```

```
64. HEAD_NAMES = [
65.     'call_offset', 'put_offset', 'wing_width', 'dte',
66.     'pop', 'roi', 'max_loss', 'confidence', 'entry', 'exit'
67. ]
68.
69. # Color schemes
70. COLORS = ['#2E86AB', '#A23B72', '#F18F01', '#C73E1D', '#3B1F2B', '#95C623']
71. CMAP_DIVERGE = 'RdBu_r'
72. CMAP_SEQ = 'viridis'
73.
74.
75. def safe_nan_to_num(X: np.ndarray) -> np.ndarray:
76.     """Replace NaN/Inf with 0."""
77.     return np.nan_to_num(X, nan=0.0, posinf=0.0, neginf=0.0).astype(np.float32)
78.
79.
80. def load_cde_model(ckpt_path, input_dim=None):
81.     """Load a CDE model from checkpoint."""
82.     if input_dim is None:
83.         input_dim = INPUT_DIM_V22
84.
85.     ckpt = torch.load(ckpt_path, map_location=DEVICE, weights_only=False)
86.
87.     seq_len = ckpt.get('seq_len', 256)
88.     config = ckpt.get('model_config', ckpt.get('config', {}))
89.     d_model = config.get('d_model', 128)
90.     n_layers = config.get('n_layers', 2)
91.     use_topk = config.get('use_topk_moe', False)
92.
93.     model = CondorBrain(
94.         d_model=d_model,
95.         n_layers=n_layers,
96.         input_dim=input_dim,
97.         use_cde=True,
98.         use_topk_moe=use_topk
99.     )
100.
101.     state_dict = ckpt['state_dict']
102.     if any(k.startswith('module.') for k in state_dict.keys()):
103.         state_dict = {k.replace('module.', ''): v for k, v in state_dict.items()}
104.
105.     model.load_state_dict(state_dict)
106.     model.to(DEVICE)
107.     model.eval()
108.
109.     return model, ckpt, seq_len
110.
111.
112. # =====
113. # CORE ANALYSIS FUNCTIONS
114. # =====
115.
116. def analyze_permutation_importance(model, X, feature_names, seq_len, n_samples=1000):
117.     """Compute feature importance via permutation."""
118.     n_samples = min(n_samples, len(X) - seq_len)
119.     indices = np.random.choice(len(X) - seq_len, n_samples, replace=False)
120.
121.     batch_X = []
122.     for idx in indices:
123.         x_seq = X[idx : idx + seq_len]
124.         if len(x_seq) == seq_len:
125.             batch_X.append(torch.tensor(x_seq, dtype=torch.float32))
126.
127.     if not batch_X:
128.         return {}
129.
130.     X_base = torch.stack(batch_X).to(DEVICE)
131.
132.     with torch.no_grad():
133.         base_out = model(X_base)
134.         if isinstance(base_out, tuple):
```



```
135.         base_out = base_out[0]
136.
137.     valid_mask = torch.isfinite(base_out).all(dim=1)
138.     n_valid = valid_mask.sum().item()
139.
140.     if n_valid == 0:
141.         return {}
142.
143.     X_base = X_base[valid_mask]
144.     base_out = base_out[valid_mask]
145.
146.     importances = {}
147.     per_head_importances = {h: {} for h in HEAD_NAMES[:base_out.shape[1]]}
148.
149.     for i, fname in enumerate(tqdm(feature_names, desc="Permutation importance",
leave=False)):
150.         X_perm = X_base.clone()
151.         perm_indices = torch.randperm(n_valid)
152.         X_perm[:, :, i] = X_perm[perm_indices, :, i]
153.
154.         with torch.no_grad():
155.             perm_out = model(X_perm)
156.             if isinstance(perm_out, tuple):
157.                 perm_out = perm_out[0]
158.
159.             # Global importance
160.             diff_tensor = torch.abs(base_out - perm_out)
161.             diff_per_sample = diff_tensor.mean(dim=1)
162.             idx_valid_perm = torch.isfinite(diff_per_sample)
163.
164.             if idx_valid_perm.sum() > 0:
165.                 diff = diff_per_sample[idx_valid_perm].mean().item()
166.             else:
167.                 diff = 0.0
168.
169.             importances[fname] = diff
170.
171.             # Per-head importance
172.             for h_idx, h_name in enumerate(HEAD_NAMES[:base_out.shape[1]]):
173.                 head_diff = torch.abs(base_out[:, h_idx] - perm_out[:, h_idx])
174.                 valid_head = torch.isfinite(head_diff)
175.                 if valid_head.sum() > 0:
176.                     per_head_importances[h_name][fname] = head_diff[valid_head].mean().item()
177.                 else:
178.                     per_head_importances[h_name][fname] = 0.0
179.
180.             # Normalize to 0-100
181.             total_impact = sum(importances.values())
182.             if total_impact > 0:
183.                 for k in importances:
184.                     importances[k] = (importances[k] / total_impact) * 100.0
185.
186.             # Normalize per-head
187.             for h_name in per_head_importances:
188.                 total = sum(per_head_importances[h_name].values())
189.                 if total > 0:
190.                     for k in per_head_importances[h_name]:
191.                         per_head_importances[h_name][k] = (per_head_importances[h_name][k] / total)
192.                 * 100.0
193.
194.     return importances, per_head_importances
195.
196. def compute_gradient_saliency(model, X, seq_len, n_samples=500):
197.     """Compute input gradient saliency maps."""
198.     n_samples = min(n_samples, len(X) - seq_len)
199.     indices = np.random.choice(len(X) - seq_len, n_samples, replace=False)
200.
201.     all_grads = []
202.
203.     model.eval()
```

```
204.     for idx in tqdm(indices, desc="Computing gradients", leave=False):
205.         seq = X[idx : idx + seq_len]
206.         x_tensor = torch.tensor(seq, device=DEVICE, dtype=torch.float32).unsqueeze(0)
207.         x_tensor.requires_grad_(True)
208.
209.         out = model(x_tensor)
210.         if isinstance(out, tuple):
211.             out = out[0]
212.
213.         # Sum all outputs for gradient
214.         loss = out.sum()
215.
216.         model.zero_grad()
217.         loss.backward()
218.
219.         if x_tensor.grad is not None:
220.             grad = x_tensor.grad.detach().cpu().numpy()[0] # (T, D)
221.             all_grads.append(grad)
222.
223.         x_tensor.requires_grad_(False)
224.
225.     if not all_grads:
226.         return None
227.
228.     # Average absolute gradient across samples
229.     grads = np.stack(all_grads) # (N, T, D)
230.     mean_abs_grad = np.mean(np.abs(grads), axis=(0, 1)) # (D,)
231.
232.     # Temporal gradient pattern
233.     temporal_grad = np.mean(np.abs(grads), axis=(0, 2)) # (T,)
234.
235.     return {
236.         'feature_saliency': mean_abs_grad,
237.         'temporal_saliency': temporal_grad,
238.         'raw_grads': grads,
239.     }
240.
241.
242. def estimate_fisher_information(model, X, seq_len, n_samples=300):
243.     """Estimate Fisher Information Matrix diagonal (parameter sensitivity)."""
244.     n_samples = min(n_samples, len(X) - seq_len)
245.     indices = np.random.choice(len(X) - seq_len, n_samples, replace=False)
246.
247.     # Collect squared gradients for each parameter
248.     fisher_diag = {name: torch.zeros_like(param) for name, param in
model.named_parameters() if param.requires_grad}
249.
250.     model.eval()
251.     for idx in tqdm(indices, desc="Estimating Fisher", leave=False):
252.         seq = X[idx : idx + seq_len]
253.         x_tensor = torch.tensor(seq, device=DEVICE, dtype=torch.float32).unsqueeze(0)
254.
255.         out = model(x_tensor)
256.         if isinstance(out, tuple):
257.             out = out[0]
258.
259.         # Use log-likelihood proxy (negative MSE from mean)
260.         log_prob = -((out - out.mean()) ** 2).sum()
261.
262.         model.zero_grad()
263.         log_prob.backward()
264.
265.         for name, param in model.named_parameters():
266.             if param.grad is not None and name in fisher_diag:
267.                 fisher_diag[name] += param.grad.detach() ** 2
268.
269.     # Average
270.     for name in fisher_diag:
271.         fisher_diag[name] /= n_samples
272.
273.     # Summarize by layer
```

```
274.     layer_fisher = {}
275.     for name, values in fisher_diag.items():
276.         layer_name = name.split('.')[0]
277.         if layer_name not in layer_fisher:
278.             layer_fisher[layer_name] = 0.0
279.             layer_fisher[layer_name] += values.sum().item()
280.
281.     return fisher_diag, layer_fisher
282.
283.
284. def estimate_hessian_spectrum(model, X, seq_len, n_samples=100, n_eigenvalues=20):
285.     """Estimate top eigenvalues of the Hessian (loss landscape curvature)."""
286.     n_samples = min(n_samples, len(X) - seq_len)
287.     indices = np.random.choice(len(X) - seq_len, n_samples, replace=False)
288.
289.     # Collect parameters
290.     params = [p for p in model.parameters() if p.requires_grad]
291.     n_params = sum(p.numel() for p in params)
292.
293.     if n_params > 100000:
294.         print(f" [Hessian] Too many params ({n_params}), using stochastic approximation")
295.         # Use Hutchinson's estimator for large models
296.         return estimate_hessian_hutchinson(model, X, seq_len, indices, n_eigenvalues)
297.
298.     # For smaller models, compute full Hessian
299.     hessian_approx = torch.zeros(n_params, n_params, device=DEVICE)
300.
301.     model.eval()
302.     for idx in tqdm(indices[:50], desc="Estimating Hessian", leave=False): # Limit for
speed
303.         seq = X[idx : idx + seq_len]
304.         x_tensor = torch.tensor(seq, device=DEVICE, dtype=torch.float32).unsqueeze(0)
305.
306.         out = model(x_tensor)
307.         if isinstance(out, tuple):
308.             out = out[0]
309.
310.         loss = ((out) ** 2).mean()
311.
312.         # First-order gradients
313.         grads = torch.autograd.grad(loss, params, create_graph=True, allow_unused=True)
314.         grads = [g if g is not None else torch.zeros_like(p) for g, p in zip(grads,
params)]
315.         grad_vec = torch.cat([g.view(-1) for g in grads])
316.
317.         # Approximate Hessian via outer product (Gauss-Newton approximation)
318.         hessian_approx += torch.outer(grad_vec, grad_vec).detach()
319.
320.     hessian_approx /= len(indices[:50])
321.
322.     # Compute eigenvalues
323.     try:
324.         eigenvalues = torch.linalg.eigvalsh(hessian_approx.cpu())
325.         eigenvalues = eigenvalues.numpy()
326.         eigenvalues = np.sort(eigenvalues)[:n_eigenvalues]
327.     except Exception as e:
328.         print(f" [Hessian] Eigenvalue computation failed: {e}")
329.         eigenvalues = np.zeros(n_eigenvalues)
330.
331.     return eigenvalues
332.
333.
334. def estimate_hessian_hutchinson(model, X, seq_len, indices, n_eigenvalues=20,
n_vectors=50):
335.     """Stochastic Hessian trace estimation using Hutchinson's method."""
336.     traces = []
337.
338.     params = [p for p in model.parameters() if p.requires_grad]
339.
340.     for _ in tqdm(range(n_vectors), desc="Hutchinson estimation", leave=False):
341.         # Random Rademacher vector
```

```
342.         v = [torch.randint_like(p, 0, 2).float() * 2 - 1 for p in params]
343.
344.         total_hvp = 0.0
345.
346.         for idx in indices[:20]:
347.             seq = X[idx : idx + seq_len]
348.             x_tensor = torch.tensor(seq, device=DEVICE, dtype=torch.float32).unsqueeze(0)
349.
350.             out = model(x_tensor)
351.             if isinstance(out, tuple):
352.                 out = out[0]
353.
354.             loss = ((out) ** 2).mean()
355.
356.             grads = torch.autograd.grad(loss, params, create_graph=True, allow_unused=True)
357.             grads = [g if g is not None else torch.zeros_like(p) for g, p in zip(grads,
params)]
358.
359.             # Hessian-vector product
360.             grad_v = sum((g * vi).sum() for g, vi in zip(grads, v))
361.             hvp = torch.autograd.grad(grad_v, params, allow_unused=True)
362.
363.             # v^T H v approximates trace
364.             for hi, vi in zip(hvp, v):
365.                 if hi is not None:
366.                     total_hvp += (hi * vi).sum().item()
367.
368.         traces.append(total_hvp / len(indices[:20]))
369.
370.         # Return trace estimate and variance
371.         trace_mean = np.mean(traces)
372.         trace_std = np.std(traces)
373.
374.         # Approximate top eigenvalue from trace (very rough)
375.         approx_eigenvalues = np.array([trace_mean / (i + 1) for i in range(n_eigenvalues)])
376.
377.         return approx_eigenvalues
378.
379.
380. def compute_shap_approximation(model, X, feature_names, seq_len, n_samples=200,
n_background=50):
381.     """Approximate SHAP values using sampling-based Shapley estimation."""
382.     n_samples = min(n_samples, len(X) - seq_len)
383.     indices = np.random.choice(len(X) - seq_len, n_samples, replace=False)
384.
385.     # Background samples for expected value
386.     bg_indices = np.random.choice(len(X) - seq_len, n_background, replace=False)
387.     background = []
388.     for idx in bg_indices:
389.         seq = X[idx : idx + seq_len]
390.         background.append(torch.tensor(seq, dtype=torch.float32))
391.     background = torch.stack(background).to(DEVICE)
392.
393.     with torch.no_grad():
394.         bg_out = model(background)
395.         if isinstance(bg_out, tuple):
396.             bg_out = bg_out[0]
397.         expected_value = bg_out.mean(dim=0).cpu().numpy()
398.
399.     # SHAP approximation via feature ablation
400.     n_features = len(feature_names)
401.     shap_values = np.zeros((n_samples, n_features, len(expected_value)))
402.
403.     model.eval()
404.     for s_idx, idx in enumerate(tqdm(indices, desc="SHAP approximation", leave=False)):
405.         seq = X[idx : idx + seq_len]
406.         x_tensor = torch.tensor(seq, device=DEVICE, dtype=torch.float32).unsqueeze(0)
407.
408.         with torch.no_grad():
409.             base_out = model(x_tensor)
410.             if isinstance(base_out, tuple):
```

```
411.         base_out = base_out[0]
412.         base_pred = base_out[0].cpu().numpy()
413.
414.         # Ablate each feature
415.         for f_idx in range(n_features):
416.             x_ablated = x_tensor.clone()
417.             # Replace with background mean
418.             x_ablated[:, :, f_idx] = background[:, :, f_idx].mean()
419.
420.             with torch.no_grad():
421.                 ablated_out = model(x_ablated)
422.                 if isinstance(ablated_out, tuple):
423.                     ablated_out = ablated_out[0]
424.                 ablated_pred = ablated_out[0].cpu().numpy()
425.
426.             # SHAP ≈ (original - ablated) contribution
427.             shap_values[s_idx, f_idx, :] = base_pred - ablated_pred
428.
429.         # Average absolute SHAP per feature
430.         mean_abs_shap = np.mean(np.abs(shap_values), axis=(0, 2)) # (n_features,)
431.
432.         # Per-head SHAP
433.         per_head_shap = {}
434.         for h_idx, h_name in enumerate(HEAD_NAMES[:shap_values.shape[2]]):
435.             per_head_shap[h_name] = np.mean(np.abs(shap_values[:, :, h_idx]), axis=0)
436.
437.         return {
438.             'mean_abs_shap': dict(zip(feature_names, mean_abs_shap)),
439.             'per_head_shap': per_head_shap,
440.             'raw_shap': shap_values,
441.             'expected_value': expected_value,
442.         }
443.
444.
445. def compute_mutual_information(X, feature_names, seq_len, n_samples=5000):
446.     """Compute pairwise mutual information between features."""
447.     n_samples = min(n_samples, len(X))
448.
449.     # Use last timestep of sequences
450.     indices = np.random.choice(len(X) - seq_len, n_samples, replace=False)
451.     data = X[indices + seq_len - 1] # (n_samples, n_features)
452.
453.     n_features = len(feature_names)
454.     mi_matrix = np.zeros((n_features, n_features))
455.
456.     # Discretize for MI calculation
457.     n_bins = 20
458.     data_discrete = np.zeros_like(data, dtype=int)
459.     for i in range(n_features):
460.         data_discrete[:, i] = np.digitize(data[:, i], np.linspace(data[:, i].min(), data[:, i].max(), n_bins))
461.
462.     for i in tqdm(range(n_features), desc="Computing MI", leave=False):
463.         for j in range(i, n_features):
464.             mi = mutual_info_score(data_discrete[:, i], data_discrete[:, j])
465.             mi_matrix[i, j] = mi
466.             mi_matrix[j, i] = mi
467.
468.     # Normalize to [0, 1]
469.     mi_max = mi_matrix.max()
470.     if mi_max > 0:
471.         mi_matrix /= mi_max
472.
473.     return mi_matrix
474.
475.
476. def train_surrogate_tree(model, X, feature_cols, seq_len, n_samples=5000, target_head=5):
477.     """Train surrogate decision tree for a specific output head."""
478.     n_samples = min(n_samples, len(X) - seq_len)
479.     idxs = np.random.randint(0, len(X) - seq_len, size=n_samples)
480.
```

```
481.     input_states = []
482.     targets = []
483.
484.     with torch.no_grad():
485.         for idx in idxs:
486.             seq = X[idx : idx + seq_len]
487.             last_step = seq[-1]
488.
489.             x_tensor = torch.tensor(seq, device=DEVICE).unsqueeze(0).float()
490.             out = model(x_tensor)
491.             if isinstance(out, tuple):
492.                 out = out[0]
493.
494.             pred = out[0, target_head].item()
495.
496.             if np.isfinite(pred):
497.                 input_states.append(last_step)
498.                 targets.append(pred)
499.
500.     if len(input_states) < 50:
501.         return None, None, None, None
502.
503.     input_states = np.array(input_states)
504.     targets = np.array(targets)
505.
506.     tree = DecisionTreeRegressor(max_depth=4, min_samples_leaf=30)
507.     tree.fit(input_states, targets)
508.
509.     r2 = tree.score(input_states, targets)
510.     rules = export_text(tree, feature_names=feature_cols, decimals=3)
511.
512.     tree_importances = dict(zip(feature_cols, tree.feature_importances_ * 100))
513.
514.     return tree, r2, rules, tree_importances
515.
516.
517. def compute_output_statistics(model, X, seq_len, n_samples=2000):
518.     """Compute output distribution statistics for each head."""
519.     n_samples = min(n_samples, len(X) - seq_len)
520.     idxs = np.random.randint(0, len(X) - seq_len, size=n_samples)
521.
522.     all_outputs = []
523.
524.     with torch.no_grad():
525.         for idx in idxs:
526.             seq = X[idx : idx + seq_len]
527.             x_tensor = torch.tensor(seq, device=DEVICE).unsqueeze(0).float()
528.             out = model(x_tensor)
529.             if isinstance(out, tuple):
530.                 out = out[0]
531.             all_outputs.append(out[0].cpu().numpy())
532.
533.     outputs = np.array(all_outputs)
534.     valid_mask = np.isfinite(outputs).all(axis=1)
535.     outputs = outputs[valid_mask]
536.
537.     stats_dict = {}
538.     for i, head_name in enumerate(HEAD_NAMES[:outputs.shape[1]]):
539.         col = outputs[:, i]
540.         stats_dict[head_name] = {
541.             'mean': float(np.mean(col)),
542.             'std': float(np.std(col)),
543.             'min': float(np.min(col)),
544.             'max': float(np.max(col)),
545.             'skew': float(stats.skew(col)),
546.             'kurtosis': float(stats.kurtosis(col)),
547.             'q25': float(np.percentile(col, 25)),
548.             'q75': float(np.percentile(col, 75)),
549.         }
550.
551.     return stats_dict, outputs
```

```
552.
553.
554. # =====
555. # DIVERGENCE & COMPARISON METRICS
556. # =====
557.
558. def compute_divergence_metrics(imp1, imp2, feature_names):
559.     """Compute mathematical divergence between two importance distributions."""
560.     v1 = np.array([imp1.get(f, 0) for f in feature_names])
561.     v2 = np.array([imp2.get(f, 0) for f in feature_names])
562.
563.     # Normalize to probability distributions
564.     v1_norm = v1 / (v1.sum() + EPS)
565.     v2_norm = v2 / (v2.sum() + EPS)
566.
567.     # Cosine similarity
568.     cos_sim = 1 - cosine(v1_norm + EPS, v2_norm + EPS)
569.
570.     # Jensen-Shannon divergence
571.     js_div = jensenshannon(v1_norm + EPS, v2_norm + EPS)
572.
573.     # Wasserstein distance (Earth Mover's Distance)
574.     wass_dist = wasserstein_distance(v1_norm, v2_norm)
575.
576.     # Spearman rank correlation
577.     rank1 = stats.rankdata(-v1)
578.     rank2 = stats.rankdata(-v2)
579.     spearman_r, spearman_p = stats.spearmanr(rank1, rank2)
580.
581.     # Kendall's Tau (rank correlation)
582.     kendall_tau, kendall_p = stats.kendalltau(rank1, rank2)
583.
584.     # Top-K agreement
585.     top5_1 = set(sorted(imp1.keys(), key=lambda k: imp1[k], reverse=True)[:5])
586.     top5_2 = set(sorted(imp2.keys(), key=lambda k: imp2[k], reverse=True)[:5])
587.     top5_overlap = len(top5_1 & top5_2) / 5.0
588.
589.     top10_1 = set(sorted(imp1.keys(), key=lambda k: imp1[k], reverse=True)[:10])
590.     top10_2 = set(sorted(imp2.keys(), key=lambda k: imp2[k], reverse=True)[:10])
591.     top10_overlap = len(top10_1 & top10_2) / 10.0
592.
593.     # Largest shifts
594.     shifts = {f: (imp1.get(f, 0) - imp2.get(f, 0)) for f in feature_names}
595.     top_shifts = sorted(shifts.items(), key=lambda x: abs(x[1]), reverse=True)[:10]
596.
597.     # Rank changes
598.     rank_dict1 = {f: r for r, f in enumerate(sorted(imp1.keys(), key=lambda k: imp1[k],
reverse=True), 1)}
599.     rank_dict2 = {f: r for r, f in enumerate(sorted(imp2.keys(), key=lambda k: imp2[k],
reverse=True), 1)}
600.     rank_changes = {f: rank_dict1.get(f, 0) - rank_dict2.get(f, 0) for f in feature_names}
601.     top_rank_changes = sorted(rank_changes.items(), key=lambda x: abs(x[1]),
reverse=True)[:10]
602.
603.     return {
604.         'cosine_similarity': cos_sim,
605.         'jensen_shannon_divergence': js_div,
606.         'wasserstein_distance': wass_dist,
607.         'spearman_correlation': spearman_r,
608.         'spearman_pvalue': spearman_p,
609.         'kendall_tau': kendall_tau,
610.         'kendall_pvalue': kendall_p,
611.         'top5_overlap': top5_overlap,
612.         'top10_overlap': top10_overlap,
613.         'largest_shifts': top_shifts,
614.         'largest_rank_changes': top_rank_changes,
615.     }
616.
617.
618. def compute_gradient_alignment(grads1, grads2):
619.     """Compute alignment between gradient saliency maps."""
```

```
620.     if grads1 is None or grads2 is None:
621.         return None
622.
623.     s1 = grads1['feature_saliency']
624.     s2 = grads2['feature_saliency']
625.
626.     # Normalize
627.     s1_norm = s1 / (np.linalg.norm(s1) + EPS)
628.     s2_norm = s2 / (np.linalg.norm(s2) + EPS)
629.
630.     # Cosine alignment
631.     alignment = np.dot(s1_norm, s2_norm)
632.
633.     # Per-feature absolute difference
634.     diff = np.abs(s1 - s2)
635.
636.     # Temporal alignment
637.     t1 = grads1['temporal_saliency']
638.     t2 = grads2['temporal_saliency']
639.     t1_norm = t1 / (np.linalg.norm(t1) + EPS)
640.     t2_norm = t2 / (np.linalg.norm(t2) + EPS)
641.     temporal_alignment = np.dot(t1_norm, t2_norm)
642.
643.     return {
644.         'feature_alignment': alignment,
645.         'temporal_alignment': temporal_alignment,
646.         'feature_diff': diff,
647.     }
648.
649.
650. def compute_stability_metrics(outputs1, outputs2):
651.     """Physics-inspired stability analysis between two models' outputs."""
652.     n = min(len(outputs1), len(outputs2))
653.     o1, o2 = outputs1[:n], outputs2[:n]
654.
655.     metrics = {}
656.
657.     for i, head_name in enumerate(HEAD_NAMES[:o1.shape[1]]):
658.         col1, col2 = o1[:, i], o2[:, i]
659.
660.         # Mean squared difference (energy)
661.         msd = np.mean((col1 - col2) ** 2)
662.
663.         # Correlation (coherence)
664.         corr, _ = stats.pearsonr(col1, col2)
665.
666.         # Variance ratio (amplitude stability)
667.         var_ratio = np.var(col1) / (np.var(col2) + EPS)
668.
669.         # Distribution shift (Kolmogorov-Smirnov test)
670.         ks_stat, ks_pval = stats.ks_2samp(col1, col2)
671.
672.         # Wasserstein distance between output distributions
673.         wass = wasserstein_distance(col1, col2)
674.
675.         # Energy distance
676.         energy_dist = 2 * np.mean(np.abs(col1[:, None] - col2[None, :])) - \
677.             np.mean(np.abs(col1[:, None] - col1[None, :])) - \
678.             np.mean(np.abs(col2[:, None] - col2[None, :]))
679.
680.         metrics[head_name] = {
681.             'msd': float(msd),
682.             'correlation': float(corr),
683.             'variance_ratio': float(var_ratio),
684.             'ks_statistic': float(ks_stat),
685.             'ks_pvalue': float(ks_pval),
686.             'wasserstein': float(wass),
687.             'energy_distance': float(energy_dist),
688.         }
689.
690.     return metrics
```



```
691.
692.
693. def compare_fisher_information(fisher1, fisher2):
694.     """Compare Fisher Information between models."""
695.     common_layers = set(fisher1.keys()) & set(fisher2.keys())
696.
697.     ratios = {}
698.     for layer in common_layers:
699.         f1 = fisher1[layer]
700.         f2 = fisher2[layer]
701.         if f2 > EPS:
702.             ratios[layer] = f1 / f2
703.         else:
704.             ratios[layer] = float('inf') if f1 > EPS else 1.0
705.
706.     return ratios
707.
708.
709. def compare_hessian_spectra(eigs1, eigs2):
710.     """Compare Hessian eigenvalue spectra."""
711.     n = min(len(eigs1), len(eigs2))
712.     e1, e2 = eigs1[:n], eigs2[:n]
713.
714.     # Spectral norm ratio (largest eigenvalue)
715.     spectral_ratio = e1[0] / (e2[0] + EPS)
716.
717.     # Trace ratio (sum of eigenvalues)
718.     trace_ratio = e1.sum() / (e2.sum() + EPS)
719.
720.     # Condition number (ratio of largest to smallest)
721.     cond1 = e1[0] / (np.abs(e1[-1]) + EPS)
722.     cond2 = e2[0] / (np.abs(e2[-1]) + EPS)
723.
724.     # Eigenvalue distribution correlation
725.     corr, _ = stats.pearsonr(e1, e2)
726.
727.     return {
728.         'spectral_ratio': spectral_ratio,
729.         'trace_ratio': trace_ratio,
730.         'condition_number_1': cond1,
731.         'condition_number_2': cond2,
732.         'eigenvalue_correlation': corr,
733.     }
734.
735.
736. # =====
737. # VISUALIZATION FUNCTIONS
738. # =====
739.
740. def plot_importance_comparison(results, feature_names, output_dir):
741.     """Plot feature importance comparison across models."""
742.     model_names = list(results['models'].keys())
743.     n_models = len(model_names)
744.
745.     # Get all importances
746.     allimps = {name: results['models'][name].get('importances', {}) for name in
model_names}
747.
748.     # Sort by first model's importance
749.     sorted_features = sorted(feature_names, key=lambda f: allimps[model_names[0]].get(f,
0), reverse=True)[:20]
750.
751.     fig, axes = plt.subplots(1, 2, figsize=(16, 8))
752.
753.     # Bar chart comparison
754.     ax = axes[0]
755.     x = np.arange(len(sorted_features))
756.     width = 0.8 / n_models
757.
758.     for i, name in enumerate(model_names):
759.         imp = allimps[name]
```

```
760.         values = [imp.get(f, 0) for f in sorted_features]
761.         ax.barh(x + i * width, values, width, label=name, color=COLORS[i % len(COLORS)],
alpha=0.8)
762.
763.         ax.set_yticks(x + width * (n_models - 1) / 2)
764.         ax.set_yticklabels(sorted_features, fontsize=9)
765.         ax.set_xlabel('Importance (%)', fontsize=11)
766.         ax.set_title('Feature Importance Comparison (Top 20)', fontsize=12, fontweight='bold')
767.         ax.legend(loc='lower right')
768.         ax.invert_yaxis()
769.         ax.grid(axis='x', alpha=0.3)
770.
771.         # Rank comparison scatter
772.         ax = axes[1]
773.         if n_models >= 2:
774.             imp1 = allimps[model_names[0]]
775.             imp2 = allimps[model_names[1]]
776.
777.             rank1 = {f: r for r, f in enumerate(sorted(imp1.keys(), key=lambda k: imp1[k],
reverse=True), 1)}
778.             rank2 = {f: r for r, f in enumerate(sorted(imp2.keys(), key=lambda k: imp2[k],
reverse=True), 1)}
779.
780.             x_ranks = [rank1.get(f, len(feature_names)) for f in feature_names]
781.             y_ranks = [rank2.get(f, len(feature_names)) for f in feature_names]
782.
783.             ax.scatter(x_ranks, y_ranks, alpha=0.6, c=COLORS[0], s=50, edgecolors='white')
784.
785.             # Add diagonal
786.             max_rank = max(max(x_ranks), max(y_ranks))
787.             ax.plot([1, max_rank], [1, max_rank], 'k--', alpha=0.5, label='Perfect agreement')
788.
789.             # Annotate top movers
790.             rank_diff = {f: abs(rank1.get(f, 0) - rank2.get(f, 0)) for f in feature_names}
791.             top_movers = sorted(rank_diff.items(), key=lambda x: x[1], reverse=True)[:5]
792.             for f, _ in top_movers:
793.                 ax.annotate(f, (rank1.get(f, 0), rank2.get(f, 0)), fontsize=8, alpha=0.8)
794.
795.             ax.set_xlabel(f'{model_names[0]} Rank', fontsize=11)
796.             ax.set_ylabel(f'{model_names[1]} Rank', fontsize=11)
797.             ax.set_title('Feature Rank Agreement', fontsize=12, fontweight='bold')
798.             ax.legend()
799.             ax.grid(alpha=0.3)
800.
801.         plt.tight_layout()
802.         plt.savefig(os.path.join(output_dir, 'importance_comparison.png'), dpi=150,
bbox_inches='tight')
803.         plt.close()
804.
805.
806. def plot_importance_heatmap(results, feature_names, output_dir):
807.     """Plot per-head importance heatmap for each model."""
808.     for name, data in results['models'].items():
809.         per_head = data.get('per_head_importances', {})
810.         if not per_head:
811.             continue
812.
813.         heads = list(per_head.keys())
814.         # Top 20 features by average importance
815.         avg_imp = {}
816.         for f in feature_names:
817.             avg_imp[f] = np.mean([per_head[h].get(f, 0) for h in heads])
818.         top_features = sorted(avg_imp.keys(), key=lambda f: avg_imp[f], reverse=True)[:20]
819.
820.         matrix = np.array([[per_head[h].get(f, 0) for h in heads] for f in top_features])
821.
822.         fig, ax = plt.subplots(figsize=(12, 10))
823.         im = ax.imshow(matrix, cmap='YlOrRd', aspect='auto')
824.
825.         ax.set_xticks(np.arange(len(heads)))
826.         ax.set_xticklabels(heads, rotation=45, ha='right', fontsize=9)
```

```
827.         ax.set_yticks(np.arange(len(top_features)))
828.         ax.set_yticklabels(top_features, fontsize=9)
829.
830.         plt.colorbar(im, ax=ax, label='Importance (%)')
831.         ax.set_title(f'{name}: Per-Head Feature Importance', fontsize=12,
fontweight='bold')
832.
833.         plt.tight_layout()
834.         plt.savefig(os.path.join(output_dir, f'importance_heatmap_{name}.png'), dpi=150,
bbox_inches='tight')
835.         plt.close()
836.
837.
838. def plot_divergence_metrics(results, output_dir):
839.     """Plot divergence metrics visualization."""
840.     if 'divergence' not in results or not results['divergence']:
841.         return
842.
843.     pairs = list(results['divergence'].keys())
844.     n_pairs = len(pairs)
845.
846.     fig, axes = plt.subplots(2, 2, figsize=(14, 12))
847.
848.     # 1. Divergence metrics bar chart
849.     ax = axes[0, 0]
850.     metrics = ['cosine_similarity', 'spearman_correlation', 'kendall_tau', 'top5_overlap',
'top10_overlap']
851.     x = np.arange(len(metrics))
852.     width = 0.8 / n_pairs
853.
854.     for i, pair in enumerate(pairs):
855.         values = [results['divergence'][pair].get(m, 0) for m in metrics]
856.         ax.bar(x + i * width, values, width, label=pair, color=COLORS[i % len(COLORS)])
857.
858.     ax.set_xticks(x + width * (n_pairs - 1) / 2)
859.     ax.set_xticklabels(['Cosine\nSim', 'Spearman\np', 'Kendall\nτ', 'Top-5\nOverlap', 'Top-
10\nOverlap'], fontsize=9)
860.     ax.set_ylabel('Score (higher = more similar)')
861.     ax.set_title('Similarity Metrics', fontsize=12, fontweight='bold')
862.     ax.legend(fontsize=8)
863.     ax.set_ylim(0, 1.1)
864.     ax.axhline(y=0.8, color='green', linestyle='--', alpha=0.5, label='Good threshold')
865.     ax.grid(axis='y', alpha=0.3)
866.
867.     # 2. Distance metrics bar chart
868.     ax = axes[0, 1]
869.     dist_metrics = ['jensen_shannon_divergence', 'wasserstein_distance']
870.     x = np.arange(len(dist_metrics))
871.
872.     for i, pair in enumerate(pairs):
873.         values = [results['divergence'][pair].get(m, 0) for m in dist_metrics]
874.         ax.bar(x + i * width, values, width, label=pair, color=COLORS[i % len(COLORS)])
875.
876.     ax.set_xticks(x + width * (n_pairs - 1) / 2)
877.     ax.set_xticklabels(['Jensen-Shannon\nDivergence', 'Wasserstein\nDistance'], fontsize=9)
878.     ax.set_ylabel('Distance (lower = more similar)')
879.     ax.set_title('Distance Metrics', fontsize=12, fontweight='bold')
880.     ax.legend(fontsize=8)
881.     ax.grid(axis='y', alpha=0.3)
882.
883.     # 3. Largest importance shifts
884.     ax = axes[1, 0]
885.     if pairs:
886.         pair = pairs[0]
887.         shifts = results['divergence'][pair].get('largest_shifts', [])
888.         if shifts:
889.             features = [s[0] for s in shifts[:10]]
890.             values = [s[1] for s in shifts[:10]]
891.             colors = ['#E74C3C' if v > 0 else '#3498DB' for v in values]
892.             ax.barh(features, values, color=colors, alpha=0.8)
893.             ax.axvline(x=0, color='black', linewidth=0.5)
```

```
894.         ax.set_xlabel('Importance Shift (%)')
895.         ax.set_title(f'Largest Importance Shifts ({pair})', fontsize=12,
fontweight='bold')
896.         ax.grid(axis='x', alpha=0.3)
897.
898.     # 4. Largest rank changes
899.     ax = axes[1, 1]
900.     if pairs:
901.         pair = pairs[0]
902.         rank_changes = results['divergence'][pair].get('largest_rank_changes', [])
903.         if rank_changes:
904.             features = [r[0] for r in rank_changes[:10]]
905.             values = [r[1] for r in rank_changes[:10]]
906.             colors = ['#27AE60' if v > 0 else '#8E44AD' for v in values]
907.             ax.barh(features, values, color=colors, alpha=0.8)
908.             ax.axvline(x=0, color='black', linewidth=0.5)
909.             ax.set_xlabel('Rank Change (positive = moved up)')
910.             ax.set_title(f'Largest Rank Changes ({pair})', fontsize=12, fontweight='bold')
911.             ax.grid(axis='x', alpha=0.3)
912.
913.     plt.tight_layout()
914.     plt.savefig(os.path.join(output_dir, 'divergence_metrics.png'), dpi=150,
bbox_inches='tight')
915.     plt.close()
916.
917.
918. def plot_output_stability(results, output_dir):
919.     """Plot output stability analysis."""
920.     if 'stability' not in results or not results['stability']:
921.         return
922.
923.     pairs = list(results['stability'].keys())
924.
925.     for pair in pairs:
926.         metrics = results['stability'][pair]
927.         heads = list(metrics.keys())
928.
929.         fig, axes = plt.subplots(2, 3, figsize=(15, 10))
930.
931.         # 1. Correlation per head
932.         ax = axes[0, 0]
933.         corrs = [metrics[h]['correlation'] for h in heads]
934.         colors = ['#27AE60' if c > 0.9 else '#F39C12' if c > 0.7 else '#E74C3C' for c in
corrs]
935.         ax.bar(heads, corrs, color=colors, alpha=0.8)
936.         ax.axhline(y=0.9, color='green', linestyle='--', alpha=0.5)
937.         ax.axhline(y=0.7, color='orange', linestyle='--', alpha=0.5)
938.         ax.set_ylabel('Correlation')
939.         ax.set_title('Output Correlation per Head', fontsize=11, fontweight='bold')
940.         ax.set_xticklabels(heads, rotation=45, ha='right', fontsize=8)
941.         ax.set_ylim(0, 1.1)
942.
943.         # 2. MSD per head
944.         ax = axes[0, 1]
945.         msds = [metrics[h]['msd'] for h in heads]
946.         ax.bar(heads, msds, color=COLORS[1], alpha=0.8)
947.         ax.set_ylabel('Mean Squared Difference')
948.         ax.set_title('Prediction Energy Difference', fontsize=11, fontweight='bold')
949.         ax.set_xticklabels(heads, rotation=45, ha='right', fontsize=8)
950.
951.         # 3. Variance ratio
952.         ax = axes[0, 2]
953.         var_ratios = [metrics[h]['variance_ratio'] for h in heads]
954.         colors = ['#27AE60' if 0.8 < v < 1.2 else '#E74C3C' for v in var_ratios]
955.         ax.bar(heads, var_ratios, color=colors, alpha=0.8)
956.         ax.axhline(y=1.0, color='black', linestyle='-', alpha=0.5)
957.         ax.axhline(y=0.8, color='green', linestyle='--', alpha=0.3)
958.         ax.axhline(y=1.2, color='green', linestyle='--', alpha=0.3)
959.         ax.set_ylabel('Variance Ratio')
960.         ax.set_title('Amplitude Stability', fontsize=11, fontweight='bold')
961.         ax.set_xticklabels(heads, rotation=45, ha='right', fontsize=8)
```

```
962.
963.     # 4. KS statistic
964.     ax = axes[1, 0]
965.     ks_stats = [metrics[h]['ks_statistic'] for h in heads]
966.     ks_pvals = [metrics[h]['ks_pvalue'] for h in heads]
967.     colors = ['#E74C3C' if p < 0.01 else '#F39C12' if p < 0.05 else '#27AE60' for p in
ks_pvals]
968.     ax.bar(heads, ks_stats, color=colors, alpha=0.8)
969.     ax.set_ylabel('KS Statistic')
970.     ax.set_title('Distribution Shift (KS Test)', fontsize=11, fontweight='bold')
971.     ax.set_xticklabels(heads, rotation=45, ha='right', fontsize=8)
972.
973.     # 5. Wasserstein distance
974.     ax = axes[1, 1]
975.     wass = [metrics[h]['wasserstein'] for h in heads]
976.     ax.bar(heads, wass, color=COLORS[3], alpha=0.8)
977.     ax.set_ylabel('Wasserstein Distance')
978.     ax.set_title('Earth Mover Distance', fontsize=11, fontweight='bold')
979.     ax.set_xticklabels(heads, rotation=45, ha='right', fontsize=8)
980.
981.     # 6. Summary radar chart
982.     ax = axes[1, 2]
983.     # Normalize metrics to [0, 1] for radar
984.     norm_corr = np.mean(corrs)
985.     norm_var = 1 - np.mean([abs(v - 1) for v in var_ratios])
986.     norm_ks = 1 - np.mean(ks_stats)
987.     norm_wass = 1 / (1 + np.mean(wass))
988.
989.     categories = ['Correlation', 'Variance\nStability', 'Distribution\nMatch',
'Wasserstein\nSimilarity']
990.     values = [norm_corr, max(0, norm_var), max(0, norm_ks), norm_wass]
991.
992.     angles = np.linspace(0, 2 * np.pi, len(categories), endpoint=False).tolist()
993.     values += values[:1]
994.     angles += angles[:1]
995.
996.     ax.plot(angles, values, 'o-', linewidth=2, color=COLORS[0])
997.     ax.fill(angles, values, alpha=0.25, color=COLORS[0])
998.     ax.set_xticks(angles[:-1])
999.     ax.set_xticklabels(categories, fontsize=9)
1000.    ax.set_ylim(0, 1)
1001.    ax.set_title('Overall Stability Score', fontsize=11, fontweight='bold')
1002.
1003.    plt.suptitle(f'Output Stability: {pair}', fontsize=14, fontweight='bold', y=1.02)
1004.    plt.tight_layout()
1005.    plt.savefig(os.path.join(output_dir, f'stability_{pair.replace(" ",
" ").replace("/", "-")}.png'),
1006.                dpi=150, bbox_inches='tight')
1007.    plt.close()
1008.
1009.
1010. def plot_gradient_saliency(results, feature_names, output_dir):
1011.     """Plot gradient saliency comparison."""
1012.     model_names = list(results['models'].keys())
1013.
1014.     fig, axes = plt.subplots(len(model_names), 2, figsize=(14, 5 * len(model_names)))
1015.     if len(model_names) == 1:
1016.         axes = axes.reshape(1, -1)
1017.
1018.     for i, name in enumerate(model_names):
1019.         grads = results['models'][name].get('gradient_saliency')
1020.         if grads is None:
1021.             continue
1022.
1023.         # Feature saliency
1024.         ax = axes[i, 0]
1025.         saliency = grads['feature_saliency']
1026.         sorted_idx = np.argsort(saliency)[::-1][:20]
1027.         sorted_features = [feature_names[j] for j in sorted_idx]
1028.         sorted_values = saliency[sorted_idx]
1029.
```

```
1030.         ax.barh(sorted_features[::-1], sorted_values[::-1], color=COLORS[i % len(COLORS)],
1031.                  alpha=0.8)
1032.         ax.set_xlabel('Mean |Gradient|')
1033.         ax.set_title(f'{name}: Feature Gradient Saliency', fontsize=11, fontweight='bold')
1034.         ax.grid(axis='x', alpha=0.3)
1035.         # Temporal saliency
1036.         ax = axes[i, 1]
1037.         temporal = grads['temporal_saliency']
1038.         ax.plot(temporal, color=COLORS[i % len(COLORS)], linewidth=1.5)
1039.         ax.fill_between(range(len(temporal)), temporal, alpha=0.3, color=COLORS[i %
1040. len(COLORS)])
1041.         ax.set_xlabel('Timestep (t)')
1042.         ax.set_ylabel('Mean |Gradient|')
1043.         ax.set_title(f'{name}: Temporal Attention Pattern', fontsize=11, fontweight='bold')
1044.         ax.grid(alpha=0.3)
1045.         # Mark recent vs. distant past
1046.         seq_len = len(temporal)
1047.         ax.axvline(x=seq_len * 0.8, color='red', linestyle='--', alpha=0.5, label='Recent
1048. 20%')
1049.         ax.legend()
1050.         plt.tight_layout()
1051.         plt.savefig(os.path.join(output_dir, 'gradient_saliency.png'), dpi=150,
1052.                    bbox_inches='tight')
1053.         plt.close()
1054.
1055. def plot_gradient_alignment(results, output_dir):
1056.     """Plot gradient alignment between models."""
1057.     if 'gradient_alignment' not in results or not results['gradient_alignment']:
1058.         return
1059.
1060.     pairs = list(results['gradient_alignment'].keys())
1061.
1062.     fig, axes = plt.subplots(1, len(pairs), figsize=(7 * len(pairs), 6))
1063.     if len(pairs) == 1:
1064.         axes = [axes]
1065.
1066.     for i, pair in enumerate(pairs):
1067.         align = results['gradient_alignment'][pair]
1068.         if align is None:
1069.             continue
1070.
1071.         ax = axes[i]
1072.
1073.         # Alignment scores
1074.         scores = {
1075.             'Feature\nAlignment': align['feature_alignment'],
1076.             'Temporal\nAlignment': align['temporal_alignment'],
1077.         }
1078.
1079.         colors = ['#27AE60' if v > 0.8 else '#F39C12' if v > 0.5 else '#E74C3C' for v in
1080. scores.values()]
1081.         ax.bar(scores.keys(), scores.values(), color=colors, alpha=0.8)
1082.         ax.axhline(y=0.8, color='green', linestyle='--', alpha=0.5, label='Good threshold')
1083.         ax.set_ylim(0, 1.1)
1084.         ax.set_ylabel('Cosine Alignment')
1085.         ax.set_title(f'Gradient Alignment: {pair}', fontsize=11, fontweight='bold')
1086.         ax.legend()
1087.         plt.tight_layout()
1088.         plt.savefig(os.path.join(output_dir, 'gradient_alignment.png'), dpi=150,
1089.                    bbox_inches='tight')
1090.         plt.close()
1091.
1092. def plot_fisher_comparison(results, output_dir):
1093.     """Plot Fisher Information comparison."""
1094.     if 'fisher_comparison' not in results or not results['fisher_comparison']:
```

```
1095.         return
1096.
1097.     pairs = list(results['fisher_comparison'].keys())
1098.
1099.     for pair in pairs:
1100.         ratios = results['fisher_comparison'][pair]
1101.         layers = list(ratios.keys())
1102.         values = list(ratios.values())
1103.
1104.         # Cap extreme values for visualization
1105.         values = [min(max(v, 0.1), 10) for v in values]
1106.
1107.         fig, ax = plt.subplots(figsize=(12, 6))
1108.
1109.         colors = ['#E74C3C' if v > 2 or v < 0.5 else '#27AE60' for v in values]
1110.         ax.bar(layers, values, color=colors, alpha=0.8)
1111.         ax.axhline(y=1.0, color='black', linestyle='-', linewidth=2, label='Equal
sensitivity')
1112.         ax.axhline(y=2.0, color='red', linestyle='--', alpha=0.5)
1113.         ax.axhline(y=0.5, color='red', linestyle='--', alpha=0.5)
1114.
1115.         ax.set_ylabel('Fisher Information Ratio (Model1 / Model2)')
1116.         ax.set_xlabel('Layer')
1117.         ax.set_title(f'Parameter Sensitivity Comparison: {pair}', fontsize=12,
fontweight='bold')
1118.         ax.set_yscale('log')
1119.         plt.xticks(rotation=45, ha='right', fontsize=8)
1120.         ax.legend()
1121.         ax.grid(axis='y', alpha=0.3)
1122.
1123.         plt.tight_layout()
1124.         plt.savefig(os.path.join(output_dir, f'fisher_comparison_{pair.replace(" ",
" _")}.png'),
1125.                     dpi=150, bbox_inches='tight')
1126.         plt.close()
1127.
1128.
1129. def plot_hessian_spectrum(results, output_dir):
1130.     """Plot Hessian eigenvalue spectrum comparison."""
1131.     model_names = list(results['models'].keys())
1132.
1133.     fig, axes = plt.subplots(1, 2, figsize=(14, 6))
1134.
1135.     # 1. Eigenvalue spectrum
1136.     ax = axes[0]
1137.     for i, name in enumerate(model_names):
1138.         eigs = results['models'][name].get('hessian_eigenvalues')
1139.         if eigs is not None:
1140.             ax.semilogy(range(1, len(eigs) + 1), np.abs(eigs) + EPS, 'o-',
1141.                         label=name, color=COLORS[i % len(COLORS)], markersize=5)
1142.
1143.     ax.set_xlabel('Eigenvalue Index')
1144.     ax.set_ylabel('|Eigenvalue| (log scale)')
1145.     ax.set_title('Hessian Eigenvalue Spectrum', fontsize=12, fontweight='bold')
1146.     ax.legend()
1147.     ax.grid(alpha=0.3)
1148.
1149.     # 2. Spectral comparison metrics
1150.     ax = axes[1]
1151.     if 'hessian_comparison' in results:
1152.         for pair, metrics in results['hessian_comparison'].items():
1153.             labels = ['Spectral\nRatio', 'Trace\nRatio', 'Eigenvalue\nCorrelation']
1154.             values = [
1155.                 min(metrics['spectral_ratio'], 10),
1156.                 min(metrics['trace_ratio'], 10),
1157.                 metrics['eigenvalue_correlation']
1158.             ]
1159.
1160.             x = np.arange(len(labels))
1161.             ax.bar(x, values, alpha=0.8, label=pair)
1162.
```

```
1163.     ax.axhline(y=1.0, color='black', linestyle='--', alpha=0.5)
1164.     ax.set_xticks(x)
1165.     ax.set_xticklabels(labels)
1166.     ax.set_ylabel('Ratio / Correlation')
1167.     ax.set_title('Loss Landscape Curvature Comparison', fontsize=12, fontweight='bold')
1168.     ax.legend()
1169.     ax.grid(axis='y', alpha=0.3)
1170.
1171.     plt.tight_layout()
1172.     plt.savefig(os.path.join(output_dir, 'hessian_spectrum.png'), dpi=150,
bbox_inches='tight')
1173.     plt.close()
1174.
1175.
1176. def plot_shap_comparison(results, feature_names, output_dir):
1177.     """Plot SHAP value comparison."""
1178.     model_names = list(results['models'].keys())
1179.
1180.     # Global SHAP comparison
1181.     fig, axes = plt.subplots(1, 2, figsize=(14, 8))
1182.
1183.     # 1. Top SHAP features bar chart
1184.     ax = axes[0]
1185.     shap_data = {}
1186.     for name in model_names:
1187.         shap = results['models'][name].get('shap_values', {})
1188.         if shap:
1189.             shap_data[name] = shap.get('mean_abs_shap', {})
1190.
1191.     if shap_data:
1192.         # Get top features from first model
1193.         first_shap = list(shap_data.values())[0]
1194.         top_features = sorted(first_shap.keys(), key=lambda k: first_shap[k],
reverse=True)[:15]
1195.
1196.         x = np.arange(len(top_features))
1197.         width = 0.8 / len(shap_data)
1198.
1199.         for i, (name, shap) in enumerate(shap_data.items()):
1200.             values = [shap.get(f, 0) for f in top_features]
1201.             ax.barh(x + i * width, values, width, label=name, color=COLORS[i %
len(COLORS)], alpha=0.8)
1202.
1203.         ax.set_yticks(x + width * (len(shap_data) - 1) / 2)
1204.         ax.set_yticklabels(top_features, fontsize=9)
1205.         ax.set_xlabel('Mean |SHAP|')
1206.         ax.set_title('SHAP Feature Attribution Comparison', fontsize=12, fontweight='bold')
1207.         ax.legend()
1208.         ax.invert_yaxis()
1209.         ax.grid(axis='x', alpha=0.3)
1210.
1211.     # 2. Per-head SHAP heatmap (first model)
1212.     ax = axes[1]
1213.     if model_names:
1214.         per_head = results['models'][model_names[0]].get('shap_values', {})
1215.         if per_head and 'per_head_shap' in per_head:
1216.             per_head_data = per_head['per_head_shap']
1217.             heads = list(per_head_data.keys())
1218.             # Top 15 features
1219.             all_shap = per_head.get('mean_abs_shap', {})
1220.             top_features = sorted(all_shap.keys(), key=lambda k: all_shap[k],
reverse=True)[:15]
1221.
1222.             matrix = np.array([[per_head_data[h][feature_names.index(f)] if f in
feature_names else 0
1223.                                for h in heads] for f in top_features])
1224.
1225.             im = ax.imshow(matrix, cmap='YlOrRd', aspect='auto')
1226.             ax.set_xticks(np.arange(len(heads)))
1227.             ax.set_xticklabels(heads, rotation=45, ha='right', fontsize=8)
1228.             ax.set_yticks(np.arange(len(top_features)))
```



```
1229.         ax.set_yticklabels(top_features, fontsize=9)
1230.         plt.colorbar(im, ax=ax, label='Mean |SHAP|')
1231.         ax.set_title(f'{model_names[0]}: Per-Head SHAP', fontsize=12,
fontweight='bold')
1232.
1233.         plt.tight_layout()
1234.         plt.savefig(os.path.join(output_dir, 'shap_comparison.png'), dpi=150,
bbox_inches='tight')
1235.         plt.close()
1236.
1237.
1238. def plot_mutual_information(results, feature_names, output_dir):
1239.     """Plot mutual information matrix."""
1240.     mi_matrix = results.get('mutual_information')
1241.     if mi_matrix is None:
1242.         return
1243.
1244.     # Select top features for readability
1245.     mi_var = np.var(mi_matrix, axis=1)
1246.     top_idx = np.argsort(mi_var)[::-1][:25]
1247.     top_features = [feature_names[i] for i in top_idx]
1248.     mi_subset = mi_matrix[np.ix_(top_idx, top_idx)]
1249.
1250.     fig, ax = plt.subplots(figsize=(12, 10))
1251.
1252.     im = ax.imshow(mi_subset, cmap='viridis', aspect='auto')
1253.     ax.set_xticks(np.arange(len(top_features)))
1254.     ax.set_xticklabels(top_features, rotation=45, ha='right', fontsize=8)
1255.     ax.set_yticks(np.arange(len(top_features)))
1256.     ax.set_yticklabels(top_features, fontsize=8)
1257.
1258.     plt.colorbar(im, ax=ax, label='Normalized Mutual Information')
1259.     ax.set_title('Feature Mutual Information Matrix', fontsize=12, fontweight='bold')
1260.
1261.     plt.tight_layout()
1262.     plt.savefig(os.path.join(output_dir, 'mutual_information.png'), dpi=150,
bbox_inches='tight')
1263.     plt.close()
1264.
1265.
1266. def plot_decision_trees(results, feature_cols, output_dir):
1267.     """Plot surrogate decision trees."""
1268.     for name, data in results['models'].items():
1269.         tree = data.get('tree_object')
1270.         if tree is None:
1271.             continue
1272.
1273.         fig, ax = plt.subplots(figsize=(20, 12))
1274.         plot_tree(tree, feature_names=feature_cols, filled=True, rounded=True,
1275.                 fontsize=8, ax=ax, proportion=True)
1276.         ax.set_title(f'{name}: Surrogate Decision Tree (ROI Head)\nR2 =
{data.get("tree_r2", 0):.3f}',
1277.                 fontsize=14, fontweight='bold')
1278.
1279.         plt.tight_layout()
1280.         plt.savefig(os.path.join(output_dir, f'decision_tree_{name}.png'), dpi=150,
bbox_inches='tight')
1281.         plt.close()
1282.
1283.
1284. def plot_output_distributions(results, output_dir):
1285.     """Plot output distribution comparison per head."""
1286.     model_names = list(results['models'].keys())
1287.
1288.     for head_idx, head_name in enumerate(HEAD_NAMES[:10]):
1289.         fig, axes = plt.subplots(1, 2, figsize=(12, 5))
1290.
1291.         # 1. Distribution histograms
1292.         ax = axes[0]
1293.         for i, name in enumerate(model_names):
1294.             outputs = results['models'][name].get('raw_outputs')
```

```
1295.         if outputs is not None and outputs.shape[1] > head_idx:
1296.             ax.hist(outputs[:, head_idx], bins=50, alpha=0.5, label=name,
1297.                    color=COLORS[i % len(COLORS)], density=True)
1298.
1299.         ax.set_xlabel(f'{head_name} Output Value')
1300.         ax.set_ylabel('Density')
1301.         ax.set_title(f'{head_name}: Output Distribution', fontsize=11, fontweight='bold')
1302.         ax.legend()
1303.         ax.grid(alpha=0.3)
1304.
1305.     # 2. Q-Q plot (if 2 models)
1306.     ax = axes[1]
1307.     if len(model_names) >= 2:
1308.         o1 = results['models'][model_names[0]].get('raw_outputs')
1309.         o2 = results['models'][model_names[1]].get('raw_outputs')
1310.         if o1 is not None and o2 is not None:
1311.             n = min(len(o1), len(o2), 1000)
1312.             q1 = np.sort(o1[:n, head_idx])
1313.             q2 = np.sort(o2[:n, head_idx])
1314.
1315.             ax.scatter(q1, q2, alpha=0.5, s=20, c=COLORS[0])
1316.             lims = [min(q1.min(), q2.min()), max(q1.max(), q2.max())]
1317.             ax.plot(lims, lims, 'k--', alpha=0.5)
1318.
1319.             ax.set_xlabel(f'{model_names[0]}')
1320.             ax.set_ylabel(f'{model_names[1]}')
1321.             ax.set_title(f'{head_name}: Q-Q Plot', fontsize=11, fontweight='bold')
1322.             ax.grid(alpha=0.3)
1323.
1324.     plt.tight_layout()
1325.     plt.savefig(os.path.join(output_dir, f'output_dist_{head_name}.png'), dpi=150,
1326.                bbox_inches='tight')
1327.     plt.close()
1328.
1329. def plot_pairwise_matrix(results, output_dir):
1330.     """Plot pairwise comparison matrices for 3+ models."""
1331.     model_names = list(results['models'].keys())
1332.     n_models = len(model_names)
1333.
1334.     if n_models < 2:
1335.         return
1336.
1337.     # Metrics to display in matrices
1338.     metrics_config = [
1339.         ('cosine_similarity', 'Cosine Similarity', 'RdYlGn', True),
1340.         ('spearman_correlation', 'Spearman Correlation', 'RdYlGn', True),
1341.         ('jensen_shannon_divergence', 'Jensen-Shannon Divergence', 'RdYlGn_r', False),
1342.         ('wasserstein_distance', 'Wasserstein Distance', 'RdYlGn_r', False),
1343.         ('top5_overlap', 'Top-5 Feature Overlap', 'RdYlGn', True),
1344.         ('top10_overlap', 'Top-10 Feature Overlap', 'RdYlGn', True),
1345.     ]
1346.
1347.     # Build matrices
1348.     matrices = {}
1349.     for metric_key, _, _, _ in metrics_config:
1350.         matrix = np.eye(n_models)
1351.         for i, m1 in enumerate(model_names):
1352.             for j, m2 in enumerate(model_names):
1353.                 if i < j:
1354.                     pair_name = f"{m1} vs {m2}"
1355.                     if pair_name not in results['divergence']:
1356.                         pair_name = f"{m2} vs {m1}"
1357.                     if pair_name in results['divergence']:
1358.                         val = results['divergence'][pair_name].get(metric_key, 0)
1359.                         matrix[i, j] = val
1360.                         matrix[j, i] = val
1361.             matrices[metric_key] = matrix
1362.
1363.     # Plot 2x3 grid of matrices
1364.     fig, axes = plt.subplots(2, 3, figsize=(18, 12))
```

```
1365.     axes = axes.flatten()
1366.
1367.     for idx, (metric_key, title, cmap, higher_better) in enumerate(metrics_config):
1368.         ax = axes[idx]
1369.         matrix = matrices[metric_key]
1370.
1371.         mask = np.eye(n_models, dtype=bool)
1372.         off_diag = matrix[~mask]
1373.         if len(off_diag) > 0:
1374.             vmin, vmax = off_diag.min(), off_diag.max()
1375.             margin = (vmax - vmin) * 0.1 + 0.01
1376.             vmin, vmax = vmin - margin, vmax + margin
1377.         else:
1378.             vmin, vmax = 0, 1
1379.
1380.         im = ax.imshow(matrix, cmap=cmap, aspect='auto', vmin=vmin, vmax=vmax)
1381.
1382.         for i in range(n_models):
1383.             for j in range(n_models):
1384.                 if i != j:
1385.                     val = matrix[i, j]
1386.                     text_color = 'white' if (higher_better and val < (vmin + vmax)/2) or \
1387.                                     (not higher_better and val > (vmin + vmax)/2)
1388.                     else 'black'
1389.                     ax.text(j, i, f'{val:.3f}', ha='center', va='center',
1390.                             fontsize=9, color=text_color, fontweight='bold')
1391.                 else:
1392.                     ax.text(j, i, '-', ha='center', va='center', fontsize=12, color='gray')
1393.
1394.         ax.set_xticks(np.arange(n_models))
1395.         ax.set_yticks(np.arange(n_models))
1396.         ax.set_xticklabels([n[:15] for n in model_names], rotation=45, ha='right',
1397.                             fontsize=9)
1398.         ax.set_yticklabels([n[:15] for n in model_names], fontsize=9)
1399.
1400.         cbar = plt.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
1401.         cbar.ax.tick_params(labelsize=8)
1402.
1403.         indicator = "↑ better" if higher_better else "↓ better"
1404.         ax.set_title(f'{title}\n({indicator})', fontsize=11, fontweight='bold')
1405.
1406.     plt.suptitle('Pairwise Model Comparison Matrices', fontsize=14, fontweight='bold',
1407.                  y=1.02)
1408.
1409.     plt.tight_layout()
1410.     plt.savefig(os.path.join(output_dir, 'pairwise_matrix.png'), dpi=150,
1411.                 bbox_inches='tight')
1412.     plt.close()
1413.
1414. def plot_model_radar_comparison(results, output_dir):
1415.     """Plot radar chart comparing all models on key metrics."""
1416.     model_names = list(results['models'].keys())
1417.     n_models = len(model_names)
1418.
1419.     if n_models < 2:
1420.         return {}
1421.
1422.     model_scores = {}
1423.
1424.     for name in model_names:
1425.         data = results['models'][name]
1426.         scores = {}
1427.
1428.         # 1. Feature balance
1429.         imp = data.get('importances', {})
1430.         if imp:
1431.             top_val = max(imp.values())
1432.             scores['Feature\nBalance'] = max(0, 1 - top_val / 50)
1433.         else:
1434.             scores['Feature\nBalance'] = 0.5
```

```
1432.         # 2. Interpretability
1433.         tree_r2 = data.get('tree_r2', 0) or 0
1434.         scores['Interpretability'] = min(tree_r2 * 2, 1.0)
1435.
1436.         # 3. Output stability
1437.         stats_data = data.get('output_stats', {})
1438.         if stats_data:
1439.             avg_skew = np.mean([abs(s.get('skew', 0)) for s in stats_data.values()])
1440.             scores['Output\nStability'] = max(0, 1 - avg_skew / 5)
1441.         else:
1442.             scores['Output\nStability'] = 0.5
1443.
1444.         # 4. Trading signal usage
1445.         trading_features = {'rsi', 'adx', 'ivr', 'vix', 'atr', 'bb_squeeze', 'stoch_k',
1446.                             'macd', 'cmf'}
1447.         if imp:
1448.             top10 = set(list(sorted(imp.keys(), key=lambda k: imp[k], reverse=True))[:10])
1449.             overlap = len(trading_features & top10)
1450.             scores['Trading\nSignals'] = overlap / 5
1451.         else:
1452.             scores['Trading\nSignals'] = 0
1453.
1454.         # 5. Gradient focus
1455.         grads = data.get('gradient_saliency')
1456.         if grads and 'temporal_saliency' in grads:
1457.             temporal = grads['temporal_saliency']
1458.             if len(temporal) > 10:
1459.                 recent = np.mean(temporal[-int(len(temporal)*0.2):])
1460.                 distant = np.mean(temporal[:int(len(temporal)*0.2)])
1461.                 balance = 1 - abs(recent - distant) / (recent + distant + 1e-6)
1462.                 scores['Temporal\nBalance'] = balance
1463.             else:
1464.                 scores['Temporal\nBalance'] = 0.5
1465.         else:
1466.             scores['Temporal\nBalance'] = 0.5
1467.
1468.         # 6. Attribution consistency
1469.         shap = data.get('shap_values')
1470.         if shap and imp:
1471.             shap_imp = shap.get('mean_abs_shap', {})
1472.             if shap_imp:
1473.                 shap_top5 = set(sorted(shap_imp.keys(), key=lambda k: shap_imp[k],
1474.                                     reverse=True)[:5])
1475.                 perm_top5 = set(sorted(imp.keys(), key=lambda k: imp[k], reverse=True)[:5])
1476.                 scores['Attribution\nConsistency'] = len(shap_top5 & perm_top5) / 5
1477.             else:
1478.                 scores['Attribution\nConsistency'] = 0.5
1479.         else:
1480.             scores['Attribution\nConsistency'] = 0.5
1481.
1482.         model_scores[name] = scores
1483.
1484.         # Create radar chart
1485.         categories = list(list(model_scores.values())[0].keys())
1486.         n_cats = len(categories)
1487.
1488.         angles = np.linspace(0, 2 * np.pi, n_cats, endpoint=False).tolist()
1489.         angles += angles[:1]
1490.
1491.         fig, ax = plt.subplots(figsize=(10, 10), subplot_kw=dict(polar=True))
1492.
1493.         for i, (name, scores) in enumerate(model_scores.items()):
1494.             values = [scores[cat] for cat in categories]
1495.             values += values[:1]
1496.
1497.             ax.plot(angles, values, 'o-', linewidth=2, label=name, color=COLORS[i %
1498.                                     len(COLORS)])
1499.             ax.fill(angles, values, alpha=0.15, color=COLORS[i % len(COLORS)])
1500.
1501.             ax.set_xticks(angles[:-1])
1502.             ax.set_xticklabels(categories, fontsize=10)
```

```
1500.     ax.set_ylim(0, 1)
1501.     ax.set_yticks([0.2, 0.4, 0.6, 0.8, 1.0])
1502.     ax.set_yticklabels(['0.2', '0.4', '0.6', '0.8', '1.0'], fontsize=8)
1503.     ax.legend(loc='upper right', bbox_to_anchor=(1.3, 1.0))
1504.
1505.     plt.title('Model Quality Radar Comparison', fontsize=14, fontweight='bold', y=1.08)
1506.     plt.tight_layout()
1507.     plt.savefig(os.path.join(output_dir, 'model_radar.png'), dpi=150, bbox_inches='tight')
1508.     plt.close()
1509.
1510.     return model_scores
1511.
1512.
1513. def plot_ranking_summary(results, output_dir):
1514.     """Plot overall model ranking summary."""
1515.     model_names = list(results['models'].keys())
1516.     n_models = len(model_names)
1517.
1518.     if n_models < 2:
1519.         return []
1520.
1521.     rankings = {}
1522.
1523.     for name in model_names:
1524.         data = results['models'][name]
1525.         score = 0
1526.         breakdown = {}
1527.
1528.         n_pros = len(data.get('pros', []))
1529.         n_cons = len(data.get('cons', []))
1530.         breakdown['Pros'] = n_pros * 2
1531.         breakdown['Cons'] = -n_cons
1532.         score += n_pros * 2 - n_cons
1533.
1534.         tree_r2 = data.get('tree_r2', 0) or 0
1535.         if tree_r2 > 0.4:
1536.             breakdown['Interpretability'] = 3
1537.             score += 3
1538.         elif tree_r2 > 0.25:
1539.             breakdown['Interpretability'] = 1
1540.             score += 1
1541.         else:
1542.             breakdown['Interpretability'] = 0
1543.
1544.         imp = data.get('importances', {})
1545.         if imp:
1546.             top = max(imp.values())
1547.             if top < 20:
1548.                 breakdown['Feature Balance'] = 2
1549.                 score += 2
1550.             elif top < 30:
1551.                 breakdown['Feature Balance'] = 1
1552.                 score += 1
1553.             else:
1554.                 breakdown['Feature Balance'] = 0
1555.
1556.         similarities = []
1557.         for other in model_names:
1558.             if other != name:
1559.                 pair1 = f"{name} vs {other}"
1560.                 pair2 = f"{other} vs {name}"
1561.                 pair = pair1 if pair1 in results.get('divergence', {}) else pair2
1562.                 if pair in results.get('divergence', {}):
1563.                     sim = results['divergence'][pair].get('cosine_similarity', 0)
1564.                     similarities.append(sim)
1565.
1566.         if similarities:
1567.             avg_sim = np.mean(similarities)
1568.             if avg_sim > 0.8:
1569.                 breakdown['Consistency'] = 2
1570.                 score += 2
```

```
1571.         elif avg_sim > 0.6:
1572.             breakdown['Consistency'] = 1
1573.             score += 1
1574.         else:
1575.             breakdown['Consistency'] = 0
1576.
1577.         rankings[name] = {'total': score, 'breakdown': breakdown}
1578.
1579.     sorted_rankings = sorted(rankings.items(), key=lambda x: x[1]['total'], reverse=True)
1580.
1581.     fig, axes = plt.subplots(1, 2, figsize=(14, 6))
1582.
1583.     # 1. Total scores bar chart
1584.     ax = axes[0]
1585.     names = [r[0] for r in sorted_rankings]
1586.     scores = [r[1]['total'] for r in sorted_rankings]
1587.     colors = [COLORS[model_names.index(n) % len(COLORS)] for n in names]
1588.
1589.     bars = ax.barh(names[::-1], scores[::-1], color=colors[::-1], alpha=0.8)
1590.     ax.set_xlabel('Total Score', fontsize=11)
1591.     ax.set_title('Overall Model Rankings', fontsize=12, fontweight='bold')
1592.     ax.grid(axis='x', alpha=0.3)
1593.
1594.     for bar, score in zip(bars, scores[::-1]):
1595.         ax.text(bar.get_width() + 0.2, bar.get_y() + bar.get_height()/2,
1596.               f'{score}', va='center', fontsize=10, fontweight='bold')
1597.
1598.     # 2. Score breakdown stacked bar
1599.     ax = axes[1]
1600.     categories = ['Pros', 'Interpretability', 'Feature Balance', 'Consistency']
1601.
1602.     x = np.arange(len(names))
1603.     bottom = np.zeros(len(names))
1604.
1605.     color_map = {'Pros': '#27AE60', 'Interpretability': '#3498DB',
1606.                  'Feature Balance': '#9B59B6', 'Consistency': '#F39C12', 'Cons': '#E74C3C'}
1607.
1608.     for cat in categories:
1609.         values = [rankings[n]['breakdown'].get(cat, 0) for n in names]
1610.         ax.bar(x, values, bottom=bottom, label=cat, color=color_map.get(cat, 'gray'),
1611.               alpha=0.8)
1612.         bottom += np.array(values)
1613.
1614.     cons_values = [rankings[n]['breakdown'].get('Cons', 0) for n in names]
1615.     ax.bar(x, cons_values, bottom=np.zeros(len(names)), label='Cons',
1616.           color=color_map['Cons'], alpha=0.8)
1617.
1618.     ax.set_xticks(x)
1619.     ax.set_xticklabels([n[:15] for n in names], rotation=45, ha='right', fontsize=9)
1620.     ax.set_ylabel('Score Contribution', fontsize=11)
1621.     ax.set_title('Score Breakdown by Category', fontsize=12, fontweight='bold')
1622.     ax.legend(loc='upper right', fontsize=9)
1623.     ax.axhline(y=0, color='black', linewidth=0.5)
1624.     ax.grid(axis='y', alpha=0.3)
1625.
1626.     plt.tight_layout()
1627.     plt.savefig(os.path.join(output_dir, 'ranking_summary.png'), dpi=150,
1628.               bbox_inches='tight')
1629.     plt.close()
1630.
1631.     return sorted_rankings
1632.
1633. # =====
1634. # TECHNICAL ASSESSMENT & PROS/CONS GENERATION (Section 11)
1635. # =====
1636. # Thresholds for deterministic rubric
1637. THRESHOLDS = {
1638.     'r2_high': 0.50,
1639.     'r2_low': 0.20,
```

```
1639.     'nf_threshold': 0.001,
1640.     'var_collapse': 0.001,
1641.     'top1_concentration': 40,
1642.     'extreme_concentration': 80,
1643.     'entropy_min': 2.5,
1644.     'spearman_drift': 0.20,
1645.     'top10_drift': 0.20,
1646.     'overlap_good': 0.60,
1647.     'corr_good': 0.90,
1648.     'corr_moderate': 0.70,
1649.     'var_ratio_low': 0.5,
1650.     'var_ratio_high': 2.0,
1651. }
1652.
1653.
1654. def generate_pros_cons(model_name, model_data, divergence_data, stability_data,
1655.     feature_names):
1656.     """
1657.     Generate deterministic pros/cons based on rubric triggers (Spec Section 11).
1658.
1659.     Returns:
1660.         dict with 'pros' and 'cons' lists
1661.     """
1662.     pros = []
1663.     cons = []
1664.
1665.     imp = model_data.get('importances', {})
1666.     tree_r2 = model_data.get('tree_r2') or 0
1667.     output_stats = model_data.get('output_stats', {})
1668.     grads = model_data.get('gradient_saliency')
1669.     shap = model_data.get('shap_values', {})
1670.
1671.     # =====
1672.     # PROS TRIGGERS
1673.     # =====
1674.
1675.     # 1. High interpretability (surrogate fidelity)
1676.     if tree_r2 >= THRESHOLDS['r2_high']:
1677.         pros.append(f"High surrogate fidelity (R2={tree_r2:.3f} >= 0.50): decision tree
1678.         rules are reliable summary")
1679.
1680.     # 2. Stable outputs
1681.     if output_stats:
1682.         all_vars = [s.get('std', 0) ** 2 for s in output_stats.values()]
1683.         nf_rate = model_data.get('nf_rate', 0)
1684.         if nf_rate <= THRESHOLDS['nf_threshold'] and all_vars and min(all_vars) >
1685.         THRESHOLDS['var_collapse']:
1686.             pros.append("Stable finite outputs with healthy variance (no collapse/explosion
1687.             detected)")
1688.
1689.     # 3. Consistent signal with peers
1690.     if divergence_data:
1691.         overlaps = [d.get('top10_overlap', 0) for d in divergence_data.values()]
1692.         if overlaps and np.mean(overlaps) >= THRESHOLDS['overlap_good']:
1693.             pros.append(f"Signal features consistent with peer checkpoints (avg top-10
1694.             overlap={np.mean(overlaps):.2f})")
1695.
1696.     # 4. Balanced attribution (not concentrated)
1697.     if imp:
1698.         top_val = max(imp.values())
1699.         imp_values = list(imp.values())
1700.         imp_probs = np.array(imp_values) / (sum(imp_values) + EPS)
1701.         imp_entropy = -np.sum(imp_probs * np.log(imp_probs + EPS))
1702.
1703.         if top_val <= THRESHOLDS['top1_concentration'] and imp_entropy >=
1704.         THRESHOLDS['entropy_min']:
1705.             pros.append(f"Attribution not overly concentrated (top feature={top_val:.1f}%,
1706.             entropy={imp_entropy:.2f})")
1707.
1708.     # 5. Good temporal gradient balance
1709.     if grads and 'temporal_saliency' in grads:
```

```
1703.         temporal = grads['temporal_saliency']
1704.         if len(temporal) > 10:
1705.             recent = np.mean(temporal[-int(len(temporal)*0.2):])
1706.             distant = np.mean(temporal[:int(len(temporal)*0.2)])
1707.             if distant > 0 and 0.3 < recent / (distant + EPS) < 3.0:
1708.                 pros.append("Balanced temporal attention: model attends to both recent and
historical context")
1709.
1710.         # 6. Trading-relevant features in top ranks
1711.         trading_features = {'rsi', 'adx', 'ivr', 'vix', 'atr', 'bb_squeeze', 'stoch_k', 'macd',
'cmf',
1712.                             'delta_pressure', 'gamma_exposure', 'volume_imbalance'}
1713.         if imp:
1714.             top10 = set(list(sorted(imp.keys(), key=lambda k: imp[k], reverse=True))[:10])
1715.             overlap = len(trading_features & set(f.lower() for f in top10))
1716.             if overlap >= 3:
1717.                 pros.append(f"Trading-relevant features prominent ({overlap} in top-10): model
learned domain signals")
1718.
1719.         # 7. SHAP-Permutation consistency
1720.         if shap and imp:
1721.             shap_imp = shap.get('mean_abs_shap', {})
1722.             if shap_imp:
1723.                 shap_top5 = set(sorted(shap_imp.keys(), key=lambda k: shap_imp[k],
reverse=True)[:5])
1724.                 perm_top5 = set(sorted(imp.keys(), key=lambda k: imp[k], reverse=True)[:5])
1725.                 consistency = len(shap_top5 & perm_top5) / 5
1726.                 if consistency >= 0.6:
1727.                     pros.append(f"Strong attribution consistency (SHAP vs Permutation top-5
overlap={consistency:.0%})")
1728.
1729.         # =====
1730.         # CONS TRIGGERS
1731.         # =====
1732.
1733.         # 1. Potential collapse
1734.         if imp:
1735.             top_val = max(imp.values())
1736.             if top_val >= THRESHOLDS['extreme_concentration']:
1737.                 cons.append(f"Possible collapse: extreme attribution concentration (top
feature={top_val:.1f}%)")
1738.
1739.         if output_stats:
1740.             all_vars = [s.get('std', 0) ** 2 for s in output_stats.values()]
1741.             if all_vars and min(all_vars) <= THRESHOLDS['var_collapse']:
1742.                 cons.append("Possible collapse: near-constant outputs detected (variance near
zero)")
1743.
1744.         # 2. Potential explosion
1745.         nf_rate = model_data.get('nf_rate', 0)
1746.         if nf_rate >= THRESHOLDS['nf_threshold']:
1747.             cons.append(f"Potential instability: non-finite output rate={nf_rate:.2%}")
1748.
1749.         if output_stats:
1750.             high_kurtosis = [s.get('kurtosis', 0) for s in output_stats.values() if
abs(s.get('kurtosis', 0)) > 10]
1751.             if high_kurtosis:
1752.                 cons.append(f"Heavy-tailed outputs detected ({len(high_kurtosis)} heads with
|kurtosis| > 10)")
1753.
1754.         # 3. Major logic drift (vs peers)
1755.         if divergence_data:
1756.             spearman_vals = [d.get('spearman_correlation', 1) for d in
divergence_data.values()]
1757.             overlap_vals = [d.get('top10_overlap', 1) for d in divergence_data.values()]
1758.
1759.             if spearman_vals and min(spearman_vals) <= THRESHOLDS['spearman_drift']:
1760.                 cons.append(f"Major logic drift: Spearman correlation dropped to
{min(spearman_vals):.2f}")
1761.
1762.             if overlap_vals and min(overlap_vals) <= THRESHOLDS['top10_drift']:
```



```
1763.         cons.append(f"Feature ranking shifted substantially (top-10
overlap={min(overlap_vals):.0%})")
1764.
1765.     # 4. Poor surrogate fidelity
1766.     if tree_r2 < THRESHOLDS['r2_low']:
1767.         cons.append(f"Surrogate tree not faithful (R2={tree_r2:.3f} < 0.20): decision
boundary is complex or temporal")
1768.
1769.     # 5. Stability concerns
1770.     if stability_data:
1771.         for pair, metrics in stability_data.items():
1772.             for head, m in metrics.items():
1773.                 if m.get('correlation', 1) < THRESHOLDS['corr_moderate']:
1774.                     cons.append(f"Output drift: {head} correlation={m['correlation']:.2f}
(below 0.70 vs peer)")
1775.                     break
1776.                     if m.get('variance_ratio', 1) < THRESHOLDS['var_ratio_low'] or
m.get('variance_ratio', 1) > THRESHOLDS['var_ratio_high']:
1777.                         cons.append(f"Amplitude instability: {head} variance
ratio={m['variance_ratio']:.2f}")
1778.                         break
1779.
1780.     # 6. Gradient instability
1781.     if grads and 'feature_saliency' in grads:
1782.         saliency = grads['feature_saliency']
1783.         if np.isnan(saliency).any() or np.isinf(saliency).any():
1784.             cons.append("Gradient instability detected: NaN/Inf in saliency maps")
1785.
1786.     return {'pros': pros, 'cons': cons}
1787.
1788.
1789. # =====
1790. # PHYSICS INTERPRETATION FUNCTIONS (Section 14)
1791. # =====
1792.
1793. def interpret_divergence(metrics):
1794.     """Interpret divergence metrics in domain terms."""
1795.     interpretations = []
1796.
1797.     cos = metrics.get('cosine_similarity', 0)
1798.     js = metrics.get('jensen_shannon_divergence', 0)
1799.     sp = metrics.get('spearman_correlation', 0)
1800.     top5 = metrics.get('top5_overlap', 0)
1801.
1802.     # Cosine similarity interpretation
1803.     if cos > 0.95:
1804.         interpretations.append(f"Near-identical importance direction (cosine={cos:.3f}):
models learned same feature weighting")
1805.     elif cos > 0.80:
1806.         interpretations.append(f"Similar importance direction (cosine={cos:.3f}): minor
reweighting of features")
1807.     elif cos > 0.50:
1808.         interpretations.append(f"Moderate directional shift (cosine={cos:.3f}): different
feature emphasis emerging")
1809.     else:
1810.         interpretations.append(f"Major directional divergence (cosine={cos:.3f}):
fundamentally different feature selection")
1811.
1812.     # Jensen-Shannon interpretation
1813.     if js < 0.1:
1814.         interpretations.append(f"Minimal information distance (JS={js:.3f}): importance
distributions nearly identical")
1815.     elif js < 0.3:
1816.         interpretations.append(f"Moderate information distance (JS={js:.3f}): some mass
shifted between features")
1817.     else:
1818.         interpretations.append(f"Large information distance (JS={js:.3f}): substantial
redistribution of importance")
1819.
1820.     # Rank correlation
1821.     if sp > 0.9:
```

```
1822.         interpretations.append(f"Rank order preserved (Spearman={sp:.3f}): feature priority  
ordering unchanged")  
1823.         elif sp > 0.6:  
1824.             interpretations.append(f"Partial rank preservation (Spearman={sp:.3f}): some  
features swapped positions")  
1825.         else:  
1826.             interpretations.append(f"Rank structure disrupted (Spearman={sp:.3f}): feature  
priorities fundamentally reordered")  
1827.  
1828.         # Top-K interpretation  
1829.         if top5 < 0.4:  
1830.             interpretations.append(f"Signal shift warning: only {top5:.0%} of top-5 features  
overlap")  
1831.  
1832.         return interpretations  
1833.  
1834.  
1835. def interpret_stability(metrics):  
1836.     """Interpret stability metrics using physics analogies."""  
1837.     interpretations = []  
1838.  
1839.     # Aggregate across heads  
1840.     avg_corr = np.mean([m.get('correlation', 0) for m in metrics.values()])  
1841.     avg_msd = np.mean([m.get('msd', 0) for m in metrics.values()])  
1842.     avg_vr = np.mean([m.get('variance_ratio', 1) for m in metrics.values()])  
1843.     avg_ks = np.mean([m.get('ks_statistic', 0) for m in metrics.values()])  
1844.  
1845.     # Coherence (correlation)  
1846.     if avg_corr > 0.95:  
1847.         interpretations.append(f"High coherence (avg correlation={avg_corr:.3f}): outputs  
move in lockstep")  
1848.     elif avg_corr > 0.80:  
1849.         interpretations.append(f"Good coherence (avg correlation={avg_corr:.3f}): outputs  
directionally aligned")  
1850.     elif avg_corr > 0.50:  
1851.         interpretations.append(f"Partial coherence (avg correlation={avg_corr:.3f}): some  
behavioral divergence")  
1852.     else:  
1853.         interpretations.append(f"Low coherence (avg correlation={avg_corr:.3f}): models  
responding differently to same inputs")  
1854.  
1855.     # Energy gap (MSD)  
1856.     if avg_msd < 0.01:  
1857.         interpretations.append(f"Minimal energy gap (MSD={avg_msd:.4f}): outputs nearly  
identical in magnitude")  
1858.     elif avg_msd < 0.1:  
1859.         interpretations.append(f"Small energy gap (MSD={avg_msd:.4f}): minor amplitude  
differences")  
1860.     else:  
1861.         interpretations.append(f"Large energy gap (MSD={avg_msd:.4f}): significant output  
magnitude differences")  
1862.  
1863.     # Amplitude stability (variance ratio)  
1864.     if 0.8 < avg_vr < 1.25:  
1865.         interpretations.append(f"Stable amplitude (variance ratio={avg_vr:.2f}): similar  
output volatility")  
1866.     elif avg_vr < 0.5:  
1867.         interpretations.append(f"Amplitude compression (variance ratio={avg_vr:.2f}): later  
model is quieter")  
1868.     elif avg_vr > 2.0:  
1869.         interpretations.append(f"Amplitude expansion (variance ratio={avg_vr:.2f}): later  
model is louder")  
1870.  
1871.     # Distribution shift (KS)  
1872.     if avg_ks < 0.05:  
1873.         interpretations.append(f"No distribution shift (KS={avg_ks:.3f}): output  
distributions match")  
1874.     elif avg_ks < 0.15:  
1875.         interpretations.append(f"Minor distribution shift (KS={avg_ks:.3f}): subtle  
statistical differences")  
1876.     else:
```

```
1877.         interpretations.append(f"Significant distribution shift (KS={avg_ks:.3f}):  
structural change in output behavior")  
1878.  
1879.         return interpretations  
1880.  
1881.  
1882. def interpret_gradients(alignment):  
1883.     """Interpret gradient alignment."""  
1884.     if alignment is None:  
1885.         return ["Gradient analysis unavailable"]  
1886.  
1887.     interpretations = []  
1888.  
1889.     feat_align = alignment.get('feature_alignment', 0)  
1890.     temp_align = alignment.get('temporal_alignment', 0)  
1891.  
1892.     if feat_align > 0.9:  
1893.         interpretations.append(f"High feature attention alignment ({feat_align:.3f}):  
models focus on same features")  
1894.     elif feat_align > 0.7:  
1895.         interpretations.append(f"Moderate feature attention alignment ({feat_align:.3f}):  
similar but not identical focus")  
1896.     else:  
1897.         interpretations.append(f"Low feature attention alignment ({feat_align:.3f}): models  
attend to different features")  
1898.  
1899.     if temp_align > 0.9:  
1900.         interpretations.append(f"Temporal attention aligned ({temp_align:.3f}): same  
recency bias")  
1901.     elif temp_align > 0.7:  
1902.         interpretations.append(f"Partially aligned temporal focus ({temp_align:.3f})")  
1903.     else:  
1904.         interpretations.append(f"Different temporal focus ({temp_align:.3f}): models weight  
history differently")  
1905.  
1906.     return interpretations  
1907.  
1908.  
1909. def interpret_hessian(comparison):  
1910.     """Interpret Hessian spectral comparison."""  
1911.     if comparison is None:  
1912.         return ["Hessian analysis unavailable"]  
1913.  
1914.     interpretations = []  
1915.  
1916.     spec_ratio = comparison.get('spectral_ratio', 1)  
1917.     trace_ratio = comparison.get('trace_ratio', 1)  
1918.     cond1 = comparison.get('condition_number_1', 1)  
1919.     cond2 = comparison.get('condition_number_2', 1)  
1920.  
1921.     if spec_ratio > 2.0:  
1922.         interpretations.append(f"Sharper loss landscape (spectral ratio={spec_ratio:.2f}):  
later model has steeper curvature")  
1923.     elif spec_ratio < 0.5:  
1924.         interpretations.append(f"Flatter loss landscape (spectral ratio={spec_ratio:.2f}):  
later model found broader minimum")  
1925.     else:  
1926.         interpretations.append(f"Similar loss curvature (spectral ratio={spec_ratio:.2f})")  
1927.  
1928.     if cond1 > 1e4 or cond2 > 1e4:  
1929.         worse = "Model 1" if cond1 > cond2 else "Model 2"  
1930.         interpretations.append(f"High condition number detected in {worse}: potential  
numerical instability")  
1931.  
1932.     return interpretations  
1933.  
1934.  
1935. def interpret_fisher(comparison):  
1936.     """Interpret Fisher Information comparison."""  
1937.     if not comparison:  
1938.         return ["Fisher analysis unavailable"]
```

```
1939.
1940.     interpretations = []
1941.
1942.     ratios = list(comparison.values())
1943.     if not ratios:
1944.         return interpretations
1945.
1946.     max_ratio = max(ratios)
1947.     min_ratio = min(ratios)
1948.
1949.     if max_ratio > 5.0:
1950.         layer = [k for k, v in comparison.items() if v == max_ratio][0]
1951.         interpretations.append(f"High parameter sensitivity in {layer}
(ratio={max_ratio:.2f}x): fragile layer")
1952.
1953.     if min_ratio < 0.2:
1954.         layer = [k for k, v in comparison.items() if v == min_ratio][0]
1955.         interpretations.append(f"Reduced sensitivity in {layer} (ratio={min_ratio:.2f}x):
layer may be undertrained")
1956.
1957.     spread = max_ratio / (min_ratio + EPS)
1958.     if spread > 10:
1959.         interpretations.append(f"Uneven sensitivity distribution (spread={spread:.1f}x):
parameter importance concentrated")
1960.     else:
1961.         interpretations.append(f"Relatively uniform sensitivity (spread={spread:.1f}x)")
1962.
1963.     return interpretations
1964.
1965.
1966. # =====
1967. # REPORT GENERATION (Section 8)
1968. # =====
1969.
1970. def format_markdown_report(results, config):
1971.     """
1972.     Generate comprehensive markdown report (Spec Section 8.1).
1973.
1974.     10 sections as specified:
1975.     1. Executive Summary
1976.     2. Configuration & Preprocessing Contract
1977.     3. Per-Model Interpretability
1978.     4. Pairwise Divergence Analysis
1979.     5. Stability Analysis (Physics)
1980.     6. Gradient Saliency Analysis
1981.     7. Hessian & Fisher Analysis
1982.     8. Decision Tree Rules
1983.     9. Pros/Cons per Model
1984.     10. Final Recommendation & Ranking
1985.     """
1986.     lines = []
1987.     timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
1988.
1989.     model_names = list(results['models'].keys())
1990.     n_models = len(model_names)
1991.
1992.     # =====
1993.     # HEADER
1994.     # =====
1995.     lines.append("# Multi-Model Neural CDE Interpretability Audit Report")
1996.     lines.append(f"\n**Generated:** {timestamp}")
1997.     lines.append(f"**Models Compared:** {n_models}")
1998.     lines.append(f"**Samples Used:** {config.get('n_samples', 'N/A')}")
1999.     lines.append(f"**Seed:** {config.get('seed', 'N/A')}")
2000.     lines.append("")
2001.
2002.     # =====
2003.     # 1. EXECUTIVE SUMMARY
2004.     # =====
2005.     lines.append("## 1. Executive Summary")
2006.     lines.append("")
```

```
2007.
2008.     # Quick verdict
2009.     rankings = results.get('rankings', [])
2010.     if rankings:
2011.         best = rankings[0][0]
2012.         lines.append(f"***Best Model:** `{best}`")
2013.
2014.         total_pros = sum(len(results['models'][n].get('pros', [])) for n in model_names)
2015.         total_cons = sum(len(results['models'][n].get('cons', [])) for n in model_names)
2016.         lines.append(f"***Overall Health:** {total_pros} strengths identified, {total_cons}
concerns flagged")
2017.
2018.     # Key divergence summary
2019.     if results.get('divergence'):
2020.         pair = list(results['divergence'].keys())[0]
2021.         div = results['divergence'][pair]
2022.         lines.append(f"\n**Key Divergence ({pair}):**")
2023.         lines.append(f"- Cosine Similarity: {div.get('cosine_similarity', 0):.3f}")
2024.         lines.append(f"- Spearman Rank Correlation: {div.get('spearman_correlation',
0):.3f}")
2025.         lines.append(f"- Top-10 Feature Overlap: {div.get('top10_overlap', 0):.0%}")
2026.
2027.     lines.append("")
2028.
2029.     # =====
2030.     # 2. CONFIGURATION & PREPROCESSING CONTRACT
2031.     # =====
2032.     lines.append("## 2. Configuration & Preprocessing Contract")
2033.     lines.append("")
2034.     lines.append("### Feature Registry")
2035.     lines.append(f"- **Input Dimension:** {config.get('input_dim', INPUT_DIM_V22)}")
2036.     lines.append(f"- **Feature Schema:** V2.2 (FEATURE_COLS_V22)")
2037.     lines.append(f"- **Total Features:** {len(config.get('feature_names', []))}")
2038.
2039.     lines.append("\n### Sequence Configuration")
2040.     for name in model_names:
2041.         seq_len = results['models'][name].get('seq_len', 'N/A')
2042.         lines.append(f"- `{name}`: seq_len = {seq_len}")
2043.
2044.     lines.append("\n### Leakage Masking")
2045.     lines.append("- `target_spot` -> 0")
2046.     lines.append("- `max_dd_60m` -> 0")
2047.
2048.     lines.append("\n### Scaling")
2049.     lines.append("- Method: Robust Scaling (median/MAD)")
2050.     lines.append("- Clip Range: [-10, +10]")
2051.     lines.append("- NaN/Inf Handling: Replace with 0")
2052.     lines.append("")
2053.
2054.     # =====
2055.     # 3. PER-MODEL INTERPRETABILITY
2056.     # =====
2057.     lines.append("## 3. Per-Model Interpretability")
2058.     lines.append("")
2059.
2060.     for name in model_names:
2061.         data = results['models'][name]
2062.         lines.append(f"### {name}")
2063.         lines.append("")
2064.
2065.         # Top features
2066.         imp = data.get('importances', {})
2067.         if imp:
2068.             lines.append("***Top 10 Features (Permutation Importance):**")
2069.             lines.append("| Rank | Feature | Importance (%) |")
2070.             lines.append("|-----|-----|-----|")
2071.             top10 = sorted(imp.items(), key=lambda x: x[1], reverse=True)[:10]
2072.             for rank, (feat, val) in enumerate(top10, 1):
2073.                 lines.append(f"| {rank} | {feat} | {val:.2f} |")
2074.             lines.append("")
2075.
```

```
2076.         # Surrogate tree
2077.         tree_r2 = data.get('tree_r2')
2078.         if tree_r2 is not None:
2079.             interp = "High" if tree_r2 >= 0.50 else "Moderate" if tree_r2 >= 0.20 else
"Low"
2080.             lines.append(f"***Surrogate Tree R2:** {tree_r2:.4f} ({interp}
interpretability)")
2081.             lines.append("")
2082.
2083.         # Output stats summary
2084.         output_stats = data.get('output_stats', {})
2085.         if output_stats:
2086.             lines.append("***Output Statistics (ROI Head):**")
2087.             if 'roi' in output_stats:
2088.                 s = output_stats['roi']
2089.                 lines.append(f"- Mean: {s['mean']:.4f}, Std: {s['std']:.4f}")
2090.                 lines.append(f"- Skewness: {s['skew']:.3f}, Kurtosis: {s['kurtosis']:.3f}")
2091.             lines.append("")
2092.
2093.         # =====
2094.         # 4. PAIRWISE DIVERGENCE ANALYSIS
2095.         # =====
2096.         lines.append("## 4. Pairwise Divergence Analysis")
2097.         lines.append("")
2098.
2099.         for pair, metrics in results.get('divergence', {}).items():
2100.             lines.append(f"### {pair}")
2101.             lines.append("")
2102.
2103.             lines.append("***Similarity Metrics:**")
2104.             lines.append(f"- Cosine Similarity: {metrics.get('cosine_similarity', 0):.4f}")
2105.             lines.append(f"- Spearman Correlation: {metrics.get('spearman_correlation', 0):.4f}
(p={metrics.get('spearman_pvalue', 1):.2e})")
2106.             lines.append(f"- Kendall Tau: {metrics.get('kendall_tau', 0):.4f}")
2107.
2108.             lines.append("\n***Distance Metrics:**")
2109.             lines.append(f"- Jensen-Shannon Divergence:
{metrics.get('jensen_shannon_divergence', 0):.4f}")
2110.             lines.append(f"- Wasserstein Distance: {metrics.get('wasserstein_distance',
0):.4f}")
2111.
2112.             lines.append("\n***Top-K Feature Overlap:**")
2113.             lines.append(f"- Top-5 Overlap: {metrics.get('top5_overlap', 0):.0%}")
2114.             lines.append(f"- Top-10 Overlap: {metrics.get('top10_overlap', 0):.0%}")
2115.
2116.             # Interpretation
2117.             interp = interpret_divergence(metrics)
2118.             lines.append("\n***Interpretation:**")
2119.             for i in interp:
2120.                 lines.append(f"- {i}")
2121.
2122.             # Largest shifts
2123.             shifts = metrics.get('largest_shifts', [])
2124.             if shifts:
2125.                 lines.append("\n***Largest Importance Shifts:**")
2126.                 lines.append("| Feature | Shift (%) |")
2127.                 lines.append("|-----|-----|")
2128.                 for feat, shift in shifts[:5]:
2129.                     lines.append(f"| {feat} | {shift:+.2f} |")
2130.
2131.             lines.append("")
2132.
2133.         # =====
2134.         # 5. STABILITY ANALYSIS (PHYSICS)
2135.         # =====
2136.         lines.append("## 5. Stability Analysis (Physics-Inspired)")
2137.         lines.append("")
2138.
2139.         for pair, metrics in results.get('stability', {}).items():
2140.             lines.append(f"### {pair}")
2141.             lines.append("")
```

```
2142.
2143.     lines.append("| Head | Correlation | MSD | Var Ratio | KS Stat | Wasserstein |")
2144.     lines.append("|-----|-----|-----|-----|-----|-----|")
2145.     for head, m in metrics.items():
2146.         lines.append(f"| {head} | {m.get('correlation', 0):.3f} | {m.get('msd', 0):.4f} | {m.get('variance_ratio', 0):.3f} | {m.get('ks_statistic', 0):.3f} | {m.get('wasserstein', 0):.4f} |")
2147.
2148.         # Interpretation
2149.         interp = interpret_stability(metrics)
2150.         lines.append("\n**Physics Interpretation:**")
2151.         for i in interp:
2152.             lines.append(f"- {i}")
2153.
2154.     lines.append("")
2155.
2156.     # =====
2157.     # 6. GRADIENT SALIENCY ANALYSIS
2158.     # =====
2159.     lines.append("## 6. Gradient Saliency Analysis")
2160.     lines.append("")
2161.
2162.     for name in model_names:
2163.         grads = results['models'][name].get('gradient_saliency')
2164.         if grads:
2165.             lines.append(f"### {name}")
2166.             saliency = grads['feature_saliency']
2167.             feature_names = config.get('feature_names', [])
2168.             if len(feature_names) == len(saliency):
2169.                 top_idx = np.argsort(saliency)[: -1][ :5]
2170.                 lines.append("**Top-5 Gradient Saliency Features:**")
2171.                 for idx in top_idx:
2172.                     lines.append(f"- {feature_names[idx]}: {saliency[idx]:.4f}")
2173.
2174.             temporal = grads['temporal_saliency']
2175.             recent_weight = np.mean(temporal[-int(len(temporal)*0.2):])
2176.             old_weight = np.mean(temporal[:int(len(temporal)*0.2)])
2177.             lines.append(f"\n**Temporal Focus:** Recent 20% weight={recent_weight:.4f}, Distant 20% weight={old_weight:.4f}")
2178.             lines.append("")
2179.
2180.         # Gradient alignment
2181.         for pair, align in results.get('gradient_alignment', {}).items():
2182.             if align:
2183.                 lines.append(f"### Gradient Alignment: {pair}")
2184.                 interp = interpret_gradients(align)
2185.                 for i in interp:
2186.                     lines.append(f"- {i}")
2187.                 lines.append("")
2188.
2189.         # =====
2190.         # 7. HESSIAN & FISHER ANALYSIS
2191.         # =====
2192.         lines.append("## 7. Hessian & Fisher Analysis")
2193.         lines.append("")
2194.
2195.         lines.append("### Hessian Eigenspectrum")
2196.         for name in model_names:
2197.             eigs = results['models'][name].get('hessian_eigenvalues')
2198.             if eigs is not None and len(eigs) > 0:
2199.                 lines.append(f"**{name}:** Top eigenvalue = {eigs[0]:.4e}, Trace approx = {sum(eigs):.4e}")
2200.             lines.append("")
2201.
2202.         for pair, comp in results.get('hessian_comparison', {}).items():
2203.             lines.append(f"**{pair}:**")
2204.             interp = interpret_hessian(comp)
2205.             for i in interp:
2206.                 lines.append(f"- {i}")
2207.             lines.append("")
2208.
```

```
2209.     lines.append("### Fisher Information")
2210.     for pair, comp in results.get('fisher_comparison', {}).items():
2211.         lines.append(f"***{pair}:**")
2212.         interp = interpret_fisher(comp)
2213.         for i in interp:
2214.             lines.append(f"- {i}")
2215.         lines.append("")
2216.
2217.     # =====
2218.     # 8. DECISION TREE RULES
2219.     # =====
2220.     lines.append("## 8. Surrogate Decision Tree Rules")
2221.     lines.append("")
2222.
2223.     for name in model_names:
2224.         rules = results['models'][name].get('tree_rules')
2225.         if rules:
2226.             lines.append(f"### {name}")
2227.             lines.append(f"***R2 Score:** {results['models'][name].get('tree_r2', 0):.4f}")
2228.             lines.append("```")
2229.             lines.append(rules[:2000] if len(rules) > 2000 else rules) # Truncate if too
long
2230.             lines.append("```")
2231.             lines.append("")
2232.
2233.     # =====
2234.     # 9. PROS/CONS PER MODEL
2235.     # =====
2236.     lines.append("## 9. Technical Assessment: Pros & Cons")
2237.     lines.append("")
2238.
2239.     for name in model_names:
2240.         data = results['models'][name]
2241.         pros = data.get('pros', [])
2242.         cons = data.get('cons', [])
2243.
2244.         lines.append(f"### {name}")
2245.         lines.append("")
2246.
2247.         lines.append("***Strengths:***")
2248.         if pros:
2249.             for p in pros:
2250.                 lines.append(f"- {p}")
2251.         else:
2252.             lines.append("- No significant strengths identified")
2253.             lines.append("")
2254.
2255.         lines.append("***Concerns:***")
2256.         if cons:
2257.             for c in cons:
2258.                 lines.append(f"- {c}")
2259.         else:
2260.             lines.append("- No significant concerns identified")
2261.             lines.append("")
2262.
2263.     # =====
2264.     # 10. FINAL RECOMMENDATION & RANKING
2265.     # =====
2266.     lines.append("## 10. Final Recommendation & Ranking")
2267.     lines.append("")
2268.
2269.     if rankings:
2270.         lines.append("### Overall Ranking")
2271.         lines.append("| Rank | Model | Score | Breakdown |")
2272.         lines.append("|-----|-----|-----|-----|")
2273.         for rank, (name, info) in enumerate(rankings, 1):
2274.             breakdown_str = ", ".join(f"{k}:{v}" for k, v in info['breakdown'].items() if v
!= 0)
2275.             lines.append(f"| {rank} | {name} | {info['total']} | {breakdown_str} |")
2276.             lines.append("")
2277.
```



```
2278.         lines.append("### Recommendation")
2279.         best = rankings[0][0]
2280.         best_data = results['models'][best]
2281.         lines.append(f"**Recommended Model:** `{best}`")
2282.         lines.append("")
2283.         lines.append("**Rationale:**")
2284.         for p in best_data.get('pros', []):3]:
2285.             lines.append(f"- {p}")
2286.
2287.         if len(rankings) > 1:
2288.             runner_up = rankings[1][0]
2289.             lines.append(f"\n**Runner-up:** `{runner_up}`")
2290.
2291.         lines.append("")
2292.         lines.append("---")
2293.         lines.append(f"*Report generated by audit_cde_comparison.py*")
2294.
2295.         return "\n".join(lines)
2296.
2297.
2298. def convert_numpy_for_json(obj):
2299.     """Recursively convert numpy types for JSON serialization."""
2300.     if isinstance(obj, np.ndarray):
2301.         return obj.tolist()
2302.     elif isinstance(obj, (np.int64, np.int32, np.int16, np.int8)):
2303.         return int(obj)
2304.     elif isinstance(obj, (np.float64, np.float32, np.float16)):
2305.         return float(obj)
2306.     elif isinstance(obj, np.bool_):
2307.         return bool(obj)
2308.     elif isinstance(obj, dict):
2309.         return {k: convert_numpy_for_json(v) for k, v in obj.items()}
2310.     elif isinstance(obj, list):
2311.         return [convert_numpy_for_json(i) for i in obj]
2312.     elif isinstance(obj, tuple):
2313.         return tuple(convert_numpy_for_json(i) for i in obj)
2314.     else:
2315.         return obj
2316.
2317.
2318. # =====
2319. # MAIN ORCHESTRATION
2320. # =====
2321.
2322. def run_audit(model_paths, data_path, n_samples=3000,
2323. output_path='reports/model_comparison.md',
2324. seed=42, skip_gradients=False, skip_hessian=False, skip_shap=False,
2325. skip_fisher=False,
2326. skip_mi=False):
2327.     """
2328.     Main orchestration function for the audit (Spec Section 10).
2329.     """
2330.     np.random.seed(seed)
2331.     torch.manual_seed(seed)
2332.
2333.     print("=" * 80)
2334.     print("MULTI-MODEL NEURAL CDE INTERPRETABILITY AUDIT")
2335.     print("=" * 80)
2336.     print(f"\nModels: {len(model_paths)}")
2337.     for p in model_paths:
2338.         print(f"  - {p}")
2339.     print(f>Data: {data_path}")
2340.     print(f>Samples: {n_samples}")
2341.     print(f>Seed: {seed}")
2342.     print("")
2343.
2344.     # =====
2345.     # LOAD DATA
2346.     # =====
2347.     print("[1/9] Loading data...")
2348.     df = pd.read_csv(data_path)
```

```
2347.     print(f"   Loaded {len(df)} rows, {len(df.columns)} columns")
2348.
2349.     feature_names = FEATURE_COLS_V22
2350.
2351.     # Select features (add missing as zeros)
2352.     X_cols = []
2353.     for col in feature_names:
2354.         if col in df.columns:
2355.             X_cols.append(df[col].values)
2356.         else:
2357.             print(f"   [WARN] Missing feature: {col} - filling with zeros")
2358.             X_cols.append(np.zeros(len(df)))
2359.
2360.     X = np.column_stack(X_cols).astype(np.float32)
2361.     X = safe_nan_to_num(X)
2362.
2363.     # Robust scaling
2364.     print("[2/9] Applying robust scaling...")
2365.     median = np.median(X, axis=0)
2366.     mad = np.median(np.abs(X - median), axis=0)
2367.     scale = 1.4826 * mad + EPS
2368.     X = (X - median) / scale
2369.     X = np.clip(X, -10, 10)
2370.     X = safe_nan_to_num(X)
2371.
2372.     # Leakage masking
2373.     leakage_cols = ['target_spot', 'max_dd_60m']
2374.     for col in leakage_cols:
2375.         if col in feature_names:
2376.             idx = feature_names.index(col)
2377.             X[:, idx] = 0
2378.             print(f"   Masked leakage column: {col}")
2379.
2380.     print(f"   Final X shape: {X.shape}")
2381.
2382.     # =====
2383.     # LOAD MODELS
2384.     # =====
2385.     print("\n[3/9] Loading models...")
2386.     models = {}
2387.     seq_lens = {}
2388.
2389.     for path in model_paths:
2390.         name = os.path.basename(path).replace('.pth', '')
2391.         model, ckpt, seq_len = load_cde_model(path)
2392.         models[name] = model
2393.         seq_lens[name] = seq_len
2394.         print(f"   Loaded {name}: seq_len={seq_len}")
2395.
2396.     # Use minimum seq_len for comparability
2397.     T_common = min(seq_lens.values())
2398.     print(f"   Using T_common = {T_common}")
2399.
2400.     # =====
2401.     # PER-MODEL ANALYSIS
2402.     # =====
2403.     print("\n[4/9] Running per-model analysis...")
2404.     results = {
2405.         'models': {},
2406.         'divergence': {},
2407.         'stability': {},
2408.         'gradient_alignment': {},
2409.         'fisher_comparison': {},
2410.         'hessian_comparison': {},
2411.         'mutual_information': None,
2412.         'rankings': [],
2413.     }
2414.
2415.     model_names = list(models.keys())
2416.
2417.     for name, model in models.items():
```

```
2418.         print(f"\n === {name} ===")
2419.         seq_len = seq_lens[name]
2420.
2421.         model_results = {'seq_len': seq_len}
2422.
2423.         # Permutation importance
2424.         print(f"      Computing permutation importance...")
2425.         imp_result = analyze_permutation_importance(model, X, feature_names, seq_len,
n_samples)
2426.         if isinstance(imp_result, tuple):
2427.             model_results['importances'], model_results['per_head_importances'] =
imp_result
2428.         else:
2429.             model_results['importances'] = imp_result
2430.             model_results['per_head_importances'] = {}
2431.
2432.         # Surrogate tree
2433.         print(f"      Training surrogate tree...")
2434.         tree, r2, rules, tree_imp = train_surrogate_tree(model, X, feature_names, seq_len,
n_samples)
2435.         model_results['tree_object'] = tree
2436.         model_results['tree_r2'] = r2
2437.         model_results['tree_rules'] = rules
2438.         model_results['tree_importances'] = tree_imp
2439.
2440.         # Output statistics
2441.         print(f"      Computing output statistics...")
2442.         output_stats, raw_outputs = compute_output_statistics(model, X, seq_len, n_samples)
2443.         model_results['output_stats'] = output_stats
2444.         model_results['raw_outputs'] = raw_outputs
2445.
2446.         # Gradient saliency
2447.         if not skip_gradients:
2448.             print(f"      Computing gradient saliency...")
2449.             grads = compute_gradient_saliency(model, X, seq_len, min(500, n_samples))
2450.             model_results['gradient_saliency'] = grads
2451.
2452.         # SHAP approximation
2453.         if not skip_shap:
2454.             print(f"      Computing SHAP approximation...")
2455.             shap = compute_shap_approximation(model, X, feature_names, seq_len, min(200,
n_samples))
2456.             model_results['shap_values'] = shap
2457.
2458.         # Fisher Information
2459.         if not skip_fisher:
2460.             print(f"      Estimating Fisher Information...")
2461.             fisher_diag, fisher_layer = estimate_fisher_information(model, X, seq_len,
min(300, n_samples))
2462.             model_results['fisher_layer'] = fisher_layer
2463.
2464.         # Hessian spectrum
2465.         if not skip_hessian:
2466.             print(f"      Estimating Hessian spectrum...")
2467.             hessian_eigs = estimate_hessian_spectrum(model, X, seq_len, min(100,
n_samples))
2468.             model_results['hessian_eigenvalues'] = hessian_eigs
2469.
2470.         results['models'][name] = model_results
2471.
2472.         # =====
2473.         # MUTUAL INFORMATION
2474.         # =====
2475.         if not skip_mi:
2476.             print(f"\n[5/9] Computing mutual information matrix...")
2477.             mi_matrix = compute_mutual_information(X, feature_names, T_common, n_samples)
2478.             results['mutual_information'] = mi_matrix
2479.         else:
2480.             print(f"\n[5/9] Skipping mutual information (--skip-mi)")
2481.
2482.         # =====
```

```
2483.     # PAIRWISE COMPARISONS
2484.     # =====
2485.     print("\n[6/9] Computing pairwise comparisons...")
2486.
2487.     for i, name1 in enumerate(model_names):
2488.         for name2 in model_names[i+1:]:
2489.             pair = f"{name1} vs {name2}"
2490.             print(f"    {pair}")
2491.
2492.             # Divergence metrics
2493.             imp1 = results['models'][name1].get('importances', {})
2494.             imp2 = results['models'][name2].get('importances', {})
2495.             div_metrics = compute_divergence_metrics(imp1, imp2, feature_names)
2496.             results['divergence'][pair] = div_metrics
2497.
2498.             # Stability metrics
2499.             out1 = results['models'][name1].get('raw_outputs')
2500.             out2 = results['models'][name2].get('raw_outputs')
2501.             if out1 is not None and out2 is not None:
2502.                 stab_metrics = compute_stability_metrics(out1, out2)
2503.                 results['stability'][pair] = stab_metrics
2504.
2505.             # Gradient alignment
2506.             if not skip_gradients:
2507.                 grad1 = results['models'][name1].get('gradient_saliency')
2508.                 grad2 = results['models'][name2].get('gradient_saliency')
2509.                 grad_align = compute_gradient_alignment(grad1, grad2)
2510.                 results['gradient_alignment'][pair] = grad_align
2511.
2512.             # Fisher comparison
2513.             if not skip_fisher:
2514.                 f1 = results['models'][name1].get('fisher_layer', {})
2515.                 f2 = results['models'][name2].get('fisher_layer', {})
2516.                 fisher_comp = compare_fisher_information(f1, f2)
2517.                 results['fisher_comparison'][pair] = fisher_comp
2518.
2519.             # Hessian comparison
2520.             if not skip_hessian:
2521.                 h1 = results['models'][name1].get('hessian_eigenvalues')
2522.                 h2 = results['models'][name2].get('hessian_eigenvalues')
2523.                 if h1 is not None and h2 is not None:
2524.                     hess_comp = compare_hessian_spectra(h1, h2)
2525.                     results['hessian_comparison'][pair] = hess_comp
2526.
2527.     # =====
2528.     # TECHNICAL ASSESSMENT
2529.     # =====
2530.     print("\n[7/9] Generating technical assessment...")
2531.
2532.     for name in model_names:
2533.         # Get divergence data for this model
2534.         div_data = {}
2535.         stab_data = {}
2536.         for pair, metrics in results['divergence'].items():
2537.             if name in pair:
2538.                 div_data[pair] = metrics
2539.         for pair, metrics in results['stability'].items():
2540.             if name in pair:
2541.                 stab_data[pair] = metrics
2542.
2543.         assessment = generate_pros_cons(name, results['models'][name], div_data, stab_data,
feature_names)
2544.         results['models'][name]['pros'] = assessment['pros']
2545.         results['models'][name]['cons'] = assessment['cons']
2546.
2547.     # =====
2548.     # VISUALIZATION
2549.     # =====
2550.     print("\n[8/9] Generating visualizations...")
2551.
2552.     output_dir = os.path.dirname(output_path)
```

```
2553.     plots_dir = os.path.join(output_dir, 'plots')
2554.     os.makedirs(plots_dir, exist_ok=True)
2555.
2556.     try:
2557.         plot_importance_comparison(results, feature_names, plots_dir)
2558.         print(" - importance_comparison.png")
2559.     except Exception as e:
2560.         print(f" [WARN] importance_comparison failed: {e}")
2561.
2562.     try:
2563.         plot_importance_heatmap(results, feature_names, plots_dir)
2564.         print(" - importance_heatmap*.png")
2565.     except Exception as e:
2566.         print(f" [WARN] importance_heatmap failed: {e}")
2567.
2568.     try:
2569.         plot_divergence_metrics(results, plots_dir)
2570.         print(" - divergence_metrics.png")
2571.     except Exception as e:
2572.         print(f" [WARN] divergence_metrics failed: {e}")
2573.
2574.     try:
2575.         plot_output_stability(results, plots_dir)
2576.         print(" - stability*.png")
2577.     except Exception as e:
2578.         print(f" [WARN] output_stability failed: {e}")
2579.
2580.     if not skip_gradients:
2581.         try:
2582.             plot_gradient_saliency(results, feature_names, plots_dir)
2583.             print(" - gradient_saliency.png")
2584.         except Exception as e:
2585.             print(f" [WARN] gradient_saliency failed: {e}")
2586.
2587.         try:
2588.             plot_gradient_alignment(results, plots_dir)
2589.             print(" - gradient_alignment.png")
2590.         except Exception as e:
2591.             print(f" [WARN] gradient_alignment failed: {e}")
2592.
2593.     if not skip_fisher:
2594.         try:
2595.             plot_fisher_comparison(results, plots_dir)
2596.             print(" - fisher_comparison*.png")
2597.         except Exception as e:
2598.             print(f" [WARN] fisher_comparison failed: {e}")
2599.
2600.     if not skip_hessian:
2601.         try:
2602.             plot_hessian_spectrum(results, plots_dir)
2603.             print(" - hessian_spectrum.png")
2604.         except Exception as e:
2605.             print(f" [WARN] hessian_spectrum failed: {e}")
2606.
2607.     if not skip_shap:
2608.         try:
2609.             plot_shap_comparison(results, feature_names, plots_dir)
2610.             print(" - shap_comparison.png")
2611.         except Exception as e:
2612.             print(f" [WARN] shap_comparison failed: {e}")
2613.
2614.     if not skip_mi:
2615.         try:
2616.             plot_mutual_information(results, feature_names, plots_dir)
2617.             print(" - mutual_information.png")
2618.         except Exception as e:
2619.             print(f" [WARN] mutual_information failed: {e}")
2620.
2621.     try:
2622.         plot_decision_trees(results, feature_names, plots_dir)
2623.         print(" - decision_tree*.png")
```

```
2624.     except Exception as e:
2625.         print(f"    [WARN] decision_trees failed: {e}")
2626.
2627.     try:
2628.         plot_output_distributions(results, plots_dir)
2629.         print("    - output_dist*.png")
2630.     except Exception as e:
2631.         print(f"    [WARN] output_distributions failed: {e}")
2632.
2633.     try:
2634.         plot_pairwise_matrix(results, plots_dir)
2635.         print("    - pairwise_matrix.png")
2636.     except Exception as e:
2637.         print(f"    [WARN] pairwise_matrix failed: {e}")
2638.
2639.     try:
2640.         model_scores = plot_model_radar_comparison(results, plots_dir)
2641.         print("    - model_radar.png")
2642.     except Exception as e:
2643.         print(f"    [WARN] model_radar failed: {e}")
2644.         model_scores = {}
2645.
2646.     try:
2647.         rankings = plot_ranking_summary(results, plots_dir)
2648.         results['rankings'] = rankings
2649.         print("    - ranking_summary.png")
2650.     except Exception as e:
2651.         print(f"    [WARN] ranking_summary failed: {e}")
2652.
2653.     # =====
2654.     # REPORT GENERATION
2655.     # =====
2656.     print("\n[9/9] Writing reports...")
2657.
2658.     config = {
2659.         'n_samples': n_samples,
2660.         'seed': seed,
2661.         'input_dim': INPUT_DIM_V22,
2662.         'feature_names': feature_names,
2663.         'model_paths': model_paths,
2664.         'data_path': data_path,
2665.         'skip_gradients': skip_gradients,
2666.         'skip_hessian': skip_hessian,
2667.         'skip_shap': skip_shap,
2668.         'skip_fisher': skip_fisher,
2669.         'skip_mi': skip_mi,
2670.     }
2671.
2672.     # Markdown report
2673.     md_report = format_markdown_report(results, config)
2674.     with open(output_path, 'w', encoding='utf-8') as f:
2675.         f.write(md_report)
2676.     print(f"    Markdown: {output_path}")
2677.
2678.     # JSON report
2679.     json_path = output_path.replace('.md', '.json')
2680.
2681.     # Prepare JSON-safe results (remove non-serializable objects)
2682.     json_results = convert_numpy_for_json(results)
2683.
2684.     # Remove tree objects (not serializable)
2685.     for name in json_results['models']:
2686.         json_results['models'][name].pop('tree_object', None)
2687.         json_results['models'][name].pop('raw_outputs', None) # Too large
2688.         if 'gradient_saliency' in json_results['models'][name]:
2689.             gs = json_results['models'][name]['gradient_saliency']
2690.             if gs:
2691.                 gs.pop('raw_grads', None) # Too large
2692.         if 'shap_values' in json_results['models'][name]:
2693.             sv = json_results['models'][name]['shap_values']
2694.             if sv:
```

```
2695.             sv.pop('raw_shap', None) # Too large
2696.
2697.     json_results['config'] = config
2698.
2699.     with open(json_path, 'w', encoding='utf-8') as f:
2700.         json.dump(json_results, f, indent=2, default=str)
2701.     print(f"  JSON: {json_path}")
2702.
2703.     # =====
2704.     # SUMMARY
2705.     # =====
2706.     print("\n" + "=" * 80)
2707.     print("AUDIT COMPLETE")
2708.     print("=" * 80)
2709.
2710.     print("\n### Quick Summary ###")
2711.     if results['rankings']:
2712.         print(f"Best Model: {results['rankings'][0][0]}")
2713.
2714.     for name in model_names:
2715.         pros = results['models'][name].get('pros', [])
2716.         cons = results['models'][name].get('cons', [])
2717.         print(f"\n{name}:")
2718.         print(f"  Pros: {len(pros)}")
2719.         for p in pros[:2]:
2720.             print(f"    + {p[:80]}...")
2721.         print(f"  Cons: {len(cons)}")
2722.         for c in cons[:2]:
2723.             print(f"    - {c[:80]}...")
2724.
2725.     print(f"\nFull report: {output_path}")
2726.     print(f"Plots: {plots_dir}/")
2727.
2728.     return results
2729.
2730.
2731. def main():
2732.     """CLI entry point."""
2733.     parser = argparse.ArgumentParser(
2734.         description="Multi-Model Neural CDE Interpretability Comparison Audit",
2735.         formatter_class=argparse.RawDescriptionHelpFormatter,
2736.         epilog="""
2737. Examples:
2738.   python intelligence/audit_cde_comparison.py \\\
2739.     --models models/epoch_1.pth models/epoch_3.pth \\\
2740.     --data data/processed/mamba_institutional_2024_1m_v22.csv \\\
2741.     --samples 3000 \\\
2742.     --output reports/model_comparison.md
2743.
2744.   python intelligence/audit_cde_comparison.py \\\
2745.     --models models/*.pth \\\
2746.     --data data/processed/dataset.csv \\\
2747.     --skip-hessian --skip-mi \\\
2748.     --output reports/quick_comparison.md
2749.     """
2750.     )
2751.
2752.     parser.add_argument('--models', nargs='+', required=True,
2753.                         help='Paths to model checkpoints (at least 2)')
2754.     parser.add_argument('--data', required=True,
2755.                         help='Path to data CSV (V2.2 features)')
2756.     parser.add_argument('--samples', type=int, default=3000,
2757.                         help='Number of samples for analysis (default: 3000)')
2758.     parser.add_argument('--output', default='reports/model_comparison.md',
2759.                         help='Output path for markdown report')
2760.     parser.add_argument('--seed', type=int, default=42,
2761.                         help='Random seed for reproducibility')
2762.     parser.add_argument('--skip-gradients', action='store_true',
2763.                         help='Skip gradient saliency analysis')
2764.     parser.add_argument('--skip-hessian', action='store_true',
2765.                         help='Skip Hessian eigenspectrum analysis')
```

```
2766.     parser.add_argument('--skip-shap', action='store_true',
2767.                           help='Skip SHAP approximation')
2768.     parser.add_argument('--skip-fisher', action='store_true',
2769.                           help='Skip Fisher Information estimation')
2770.     parser.add_argument('--skip-mi', action='store_true',
2771.                           help='Skip mutual information matrix')
2772.
2773.     args = parser.parse_args()
2774.
2775.     # Validate inputs
2776.     if len(args.models) < 2:
2777.         parser.error("At least 2 model paths required")
2778.
2779.     for path in args.models:
2780.         if not os.path.exists(path):
2781.             parser.error(f"Model not found: {path}")
2782.
2783.     if not os.path.exists(args.data):
2784.         parser.error(f>Data file not found: {args.data}")
2785.
2786.     # Create output directory
2787.     os.makedirs(os.path.dirname(args.output) or '.', exist_ok=True)
2788.
2789.     # Run audit
2790.     run_audit(
2791.         model_paths=args.models,
2792.         data_path=args.data,
2793.         n_samples=args.samples,
2794.         output_path=args.output,
2795.         seed=args.seed,
2796.         skip_gradients=args.skip_gradients,
2797.         skip_hessian=args.skip_hessian,
2798.         skip_shap=args.skip_shap,
2799.         skip_fisher=args.skip_fisher,
2800.         skip_mi=args.skip_mi,
2801.     )
2802.
2803.
2804. if __name__ == '__main__':
2805.     main()
```

Colab – using T4 GPU

```
!git pull
!python intelligence/audit_cde_comparison.py \
--models models/epoch_1_012926.pth models/epoch_2_012926.pth models/epoch_3_013026.pth \
--data /content/spy-iron-condor-trading/data/processed/mamba_institutional_2024_1m_v22.csv \
--samples 3000 \
--output reports/model_comparison.md
```

Running:

MULTI-MODEL NEURAL CDE INTERPRETABILITY AUDIT

Models: 3

- models/epoch_1_012926.pth
- models/epoch_2_012926.pth
- models/epoch_3_013026.pth

Data: /content/spy-iron-condor-trading/data/processed/mamba_institutional_2024_1m_v22.csv

Samples: 3000

Seed: 42

[1/9] Loading data...

The script prints status updates for each analysis phase:

- Feature importance extraction
- Divergence metrics
- Gradient saliency
- SHAP approximation
- Hessian spectrum (slowest)
- Output distributions
- Plot generation

--output reports/model_comparison.md

Output locations:

- Markdown report: reports/model_comparison.md
- JSON report: reports/model_comparison.json
- All plots: reports/plots/

16 plot files generated:

reports/plots/

- └─ importance_comparison.png
- └─ importance_heatmap_epoch_1_012926.png
- └─ importance_heatmap_epoch_3_013026.png
- └─ divergence_metrics.png
- └─ stability_epoch_1_012926_vs_epoch_3_013026.png
- └─ gradient_saliency.png
- └─ gradient_alignment.png
- └─ fisher_comparison_epoch_1_012926_vs_epoch_3_013026.png
- └─ hessian_spectrum.png
- └─ shap_comparison.png
- └─ mutual_information.png
- └─ decision_tree_epoch_1_012926.png
- └─ decision_tree_epoch_3_013026.png
- └─ output_dist_*.png (10 files, one per head)
- └─ pairwise_matrix.png
- └─ model_radar.png
- └─ ranking_summary.png

Many of the advanced features that are being used are new and have never been used before. For this reason this is a work in progress and still requires much testing, however coupling various fields of mathematics, intense computational physics, entry / exit trading logistics, risk management, and neural CDE models, which integrate in real-time with multiple market predictor algorithms for diffusion, and conservative and protective professional trading rules all of which funnels into a sophisticated, and unique fuzzy logic approach for benchmarking and position sizing. This also includes neural net models, which has been computed using the *neural CDE* framework with 52 columns of data as inputs including dynamic features, Greeks, OHLCV candlesticks for m1, m5 and m14 timeframes, to enable multi-timeframe consensus. And it does not stop there. I'm just starting to work on adding literally billions of permutations of inequalities which will greatly enhance the learning process, and yield AI-NN model files like never before.

If you have any questions, comments, or need guidance on how to set this up correctly, please email to: jerryamyluv@gmail.com

